

GUIDE EXPLICATIF DU CODE

Explication détaillée, cellule par cellule

Auditeur de Cohérence Médicale - Vérification automatique de la cohérence entre
une radiographie thoracique et son compte rendu

À qui s'adresse ce guide. Ce document explique le code du projet, cellule par cellule, dans un langage accessible à une personne débutante. Pour chaque cellule, vous trouverez ce qu'elle fait, le code exact, puis une explication détaillée. Les encadrés signalent les notions clés à bien comprendre.

Le projet en bref. Dans un service de radiologie, une radiographie thoracique et son compte rendu cheminent séparément dans le système d'information ; un compte rendu peut, par erreur, être associé au mauvais examen. Ce projet construit un outil d'audit automatique qui reçoit une paire (image, texte) et estime la probabilité qu'ils décrivent réellement le même examen. Il ne s'agit pas de poser un diagnostic, mais de fournir un filet de sécurité qui signale les paires suspectes pour une revue humaine. La tâche est traitée comme une classification binaire : cohérent (1) ou incohérent (0).

Structure du guide. Partie A : construction du jeu de données (notebook de préparation, 11 cellules). Partie B : modélisation avec l'encodeur BiomedCLIP, quatre approches de complexité croissante (33 cellules). Partie C : modélisation avec l'encodeur BioViL-T, spécialisé pour la radiographie thoracique, qui aboutit au modèle finalement retenu (25 cellules).

Périmètre couvert. Ce guide documente les trois notebooks qui constituent le pipeline officiel du projet, dans leur ordre d'exécution : la construction du jeu de données, puis les deux notebooks de modélisation. Des notebooks d'exploration antérieurs, qui ont permis de tester différentes pistes avant de converger vers cette version, ne sont pas repris ici.

PARTIE A

Construction du jeu de données

Notebook : *create_dataset_sample*. Ce notebook part du jeu de données public CheXpert Plus (radiographies thoraciques et comptes rendus associés) et construit l'échantillon équilibré de 36 000 paires qui servira à entraîner et évaluer tous les modèles. C'est l'étape la plus délicate du projet : les paires cohérentes existent déjà dans les données source, mais les paires incohérentes doivent être fabriquées de toutes pièces, de manière réaliste.

Cellule 1 - Importation des bibliothèques

CE QUE FAIT CETTE CELLULE

Cette première cellule charge les outils logiciels nécessaires à la construction du jeu de données.

LE CODE

```
import pandas as pd
import numpy as np
import random
import json
import tqdm
```

EXPLICATION DÉTAILLÉE

- **pandas (pd)** est la bibliothèque de référence pour manipuler des tableaux de données (dataframes) : lecture de fichiers CSV, sélection de colonnes, fusion de tables, etc.
- **numpy (np)** fournit les outils de calcul numérique vectorisé, utilisés plus loin pour comparer rapidement les pathologies de deux dossiers sous forme de tableaux de nombres.
- **random** est le module aléatoire standard de Python ; il sera utilisé pour parcourir le réservoir de rapports dans un ordre différent à chaque patient, lors de la fabrication des incohérences.
- **json** est présent pour la lecture de fichiers structurés, dont s'appuie indirectement la lecture du fichier d'étiquettes de pathologies un peu plus loin.
- **tqdm** est une bibliothèque d'affichage de barres de progression, utile pour suivre visuellement l'avancement des boucles longues.

À RETENIR - Reproductibilité

Tout au long de ce notebook, les tirages aléatoires sont systématiquement fixés par un paramètre `random_state=42` (ou une graine équivalente). Cela garantit que deux exécutions du notebook produisent exactement le même jeu de données, ce qui est indispensable pour pouvoir comparer des résultats d'une session à l'autre.

Cellule 2 - Chargement du fichier source et sélection des colonnes utiles

CE QUE FAIT CETTE CELLULE

Cette cellule charge le fichier CSV principal de CheXpert Plus, le trie pour obtenir un ordre stable, puis ne conserve que les colonnes nécessaires au projet.

LE CODE

```
# Open the dataset containing the reports and the paths to images
pd.options.display.max_columns = None
dataset = pd.read_csv("../CheXpert/df_chexpert_plus_240401.csv")
dataset = dataset.sort_values(by="path_to_image", ignore_index=True)
# "report" column contains the full raw doctor's report
# Keeping columns with medical content: "section_findings", "section_impression",
# "section_summary" in case of future usage
columns_keep = ['path_to_image', 'deid_patient_id', 'report', 'section_findings',
                'section_impression', 'section_summary']
dataset = dataset[columns_keep]
dataset
```

EXPLICATION DÉTAILLÉE

- **pd.options.display.max_columns = None** demande à pandas d'afficher toutes les colonnes d'un dataframe sans les tronquer, ce qui facilite l'exploration visuelle dans le notebook.
- **pd.read_csv(...)** charge le fichier df_chexpert_plus_240401.csv, qui contient pour chaque radiographie son chemin d'image et le compte rendu rédigé par le radiologue.
- **sort_values(by="path_to_image", ignore_index=True)** trie les lignes par chemin d'image. Cet ordre stable est important car les tirages aléatoires suivants s'appuient sur la position des lignes dans le tableau : sans ce tri, le même random_state donnerait un résultat différent à chaque exécution.
- **columns_keep** ne retient que les colonnes utiles : path_to_image (le lien vers l'image), deid_patient_id (l'identifiant patient anonymisé), report (le compte rendu complet), et les sous-sections du rapport (constatations, impression, résumé), conservées au cas où elles seraient utiles plus tard.

À RETENIR - Désidentification des données

L'identifiant deid_patient_id est déjà anonymisé par les fournisseurs du jeu de données CheXpert avant sa mise à disposition publique. Le projet ne tente à aucun moment de réidentifier un patient ; ces données de santé sont utilisées exclusivement dans un cadre de recherche et de formation.

Cellule 3 - Une seule image par patient, puis tirage des cas positifs

CE QUE FAIT CETTE CELLULE

Pour éviter toute fuite de données, on ne garde qu'une seule radiographie par patient, tirée au hasard ; un échantillon de 20 000 patients est ensuite tiré pour former les futures paires cohérentes.

LE CODE

```
sub_dataset = dataset.groupby(by="deid_patient_id").sample(random_state=42)

# Pick randomly a subset of 20000 for the positive cases (the matches)
df_pos = sub_dataset.sample(20000, random_state=42)
```

EXPLICATION DÉTAILLÉE

- **groupby(by="deid_patient_id").sample(random_state=42)** regroupe les lignes par patient, puis tire au hasard une seule ligne (une seule radiographie) dans chaque groupe. Certains patients ont en effet plusieurs examens dans le jeu de données d'origine.

- **df_pos = sub_dataset.sample(20000, ...)** tire ensuite 20 000 patients au hasard parmi cette liste réduite à une image chacun. Ces 20 000 paires (image, rapport) sont par construction cohérentes : il s'agit de l'image et du rapport réels du même patient. Elles formeront la base des exemples positifs, étiquette cible égale à 1.

À RETENIR - Pourquoi une seule image par patient ?

Garder plusieurs radiographies d'un même patient risquerait de laisser le modèle apprendre à reconnaître la morphologie propre à ce patient, plutôt que d'apprendre la cohérence entre une image et un texte. Cette règle protège également la vie privée en appliquant un principe de minimisation des données.

Cellule 4 - Préparation du réservoir pour les cas négatifs

CE QUE FAIT CETTE CELLULE

Cette cellule prépare les données nécessaires à la fabrication des paires incohérentes : un échantillon de 20 000 patients distincts des positifs, et un grand réservoir de rapports dans lequel piocher de faux comptes rendus.

LE CODE

```
# Pick randomly a subset of 20000 for the negative cases (the mismatches)
df_neg = sub_dataset.drop(index=df_pos.index).sample(20000, random_state=42)

# Dataframe where we look for mismatches
df_build_neg = dataset.drop(index=df_pos.index)
```

EXPLICATION DÉTAILLÉE

- **df_neg** tire 20 000 patients parmi ceux qui n'ont pas été sélectionnés comme positifs. Leur radiographie reste la véritable image du patient ; seul le compte rendu associé sera remplacé, à l'étape suivante, par un rapport ne lui correspondant pas.
- **df_build_neg** constitue le réservoir de rapports candidats : l'ensemble complet des données, privé des 20 000 patients positifs. C'est dans ce réservoir que l'algorithme ira chercher, pour chaque patient négatif, un rapport approprié à lui substituer.

Cellule 5 - Chargement des étiquettes de pathologies

CE QUE FAIT CETTE CELLULE

Cette cellule charge le fichier qui indique, pour chaque radiographie, la présence ou l'absence de chaque pathologie, et associe ces étiquettes aux deux dataframes préparés précédemment.

LE CODE

```
# Exploit impression_fixed.json file: for each report, pathologies are labelled
# 1: present, 0: absent, NaN: non mentioned, -1: uncertain
impression = pd.read_json("./CheXpert/chexbert_labels/impression_fixed.json", lines=True)
impression = impression.sort_values(by="path_to_image", ignore_index=True)
# Fill NaN with numerical value 3
impression = impression.fillna(3)
# Adding the pathologies columns to dataframes df_neg and df_build_neg
```

```
df_neg = df_neg.merge(right=impression, on="path_to_image")
df_build_neg = df_build_neg.merge(right=impression, on="path_to_image")
```

EXPLICATION DÉTAILLÉE

- **pd.read_json(..., lines=True)** lit le fichier impression_fixed.json, au format JSON Lines (une ligne de texte JSON par radiographie). Ce fichier est issu d'un outil d'étiquetage automatique qui détecte, dans le texte du compte rendu, la mention de chaque pathologie.
- Le codage retenu est le suivant : 1 si la pathologie est présente, 0 si elle est explicitement absente, une valeur manquante (NaN) si elle n'est pas mentionnée du tout, et -1 si elle est évoquée de manière incertaine.
- **fillna(3)** remplace les valeurs manquantes par le code numérique 3, afin de pouvoir manipuler des tableaux numpy entièrement numériques dans les étapes suivantes, sans valeur manquante.
- **merge(right=impression, on="path_to_image")** associe ces étiquettes de pathologies à df_neg et à df_build_neg, en les alignant sur le chemin d'image, qui sert d'identifiant unique d'examen tout au long du projet.

À RETENIR - Le fichier d'étiquettes CheXbert

Les étiquettes de pathologies ne sont pas saisies manuellement : elles proviennent d'un outil d'analyse automatique du texte des comptes rendus (CheXbert), qui détecte la mention de chaque pathologie et son statut. Ce fichier sert ici uniquement à guider la fabrication des incohérences ; il n'est pas utilisé comme entrée des modèles eux-mêmes.

Cellule 6 - La fonction qui fabrique les incohérences

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction centrale du notebook : pour chaque patient négatif, elle recherche dans le réservoir un rapport dont les pathologies diffèrent du sien, selon une logique à deux dimensions (le niveau de difficulté de l'incohérence, et la proximité clinique des pathologies échangées), tout en respectant des quotas précis pour chaque combinaison.

LE CODE

```
# Function that generates the mismatches
def generate_cluster_balanced_mismatch(df_neg, df_build_neg, total_samples=20000):
    df_neg = df_neg.reset_index(drop=True)
    df_build_neg = df_build_neg.reset_index(drop=True)

    pure_pathologies = [
        'Enlarged Cardiomeastinum', 'Cardiomegaly', 'Lung Opacity', 'Lung Lesion',
        'Edema', 'Consolidation', 'Pneumonia', 'Atelectasis', 'Pneumothorax',
        'Pleural Effusion', 'Pleural Other', 'Fracture'
    ]

    # Define clinical clusters mapping index positions to cluster groups
    # C1: Heart, C2: Opacities, C3: Pleural space, C4: Focal/Bone
    cluster_mapping = {
        0: 1, 1: 1, # Enlarged Cardiomeastinum, Cardiomegaly
        2: 2, 4: 2, 5: 2, 6: 2, 7: 2, # Lung Opacity, Edema, Consolidation, Pneumonia,
        Atelectasis
```

```

8: 3, 9: 3, 10: 3, # Pneumothorax, Pleural Effusion, Pleural Other
3: 4, 11: 4 # Lung Lesion, Fracture
}

matrix_neg = df_neg[pure_pathologies].values

# Multi-dimensional sub-quotas
quotas = {
    # Total (different cluster) 40%
    ("Total", "Diff"): int(total_samples * 0.4),
    # Solid (different cluster) 24%
    ("Solid", "Diff"): int(total_samples * 0.24),
    # Easy (different cluster) 16%
    ("Easy", "Diff"): int(total_samples * 0.16),
    # Same cluster whatever the type (Total or Solid or Easy) 20%
    ("All", "Same"): int(total_samples * 0.2)
}

counts = {k: 0 for k in quotas.keys()}
mismatch_rows = []

for i in range(len(matrix_neg)):
    if sum(counts.values()) >= total_samples:
        break

    matrix_build_neg = df_build_neg[pure_pathologies].values
    len_j = len(df_build_neg)
    vec_i = matrix_neg[i]
    pos_i = (vec_i == 1)

    start_j = random.randint(0, len_j - 1)

    for offset in range(len_j):
        j = (start_j + offset) % len_j

        vec_j = matrix_build_neg[j]
        pos_j = (vec_j == 1)

        total_i_to_j = np.all(vec_j[pos_i] == 0) if np.any(pos_i) else True
        total_j_to_i = np.all(vec_i[pos_j] == 0) if np.any(pos_j) else True
        is_total = total_i_to_j and total_j_to_i and (np.any(pos_i) or np.any(pos_j))
        has_diff = np.any((vec_i == 1) & (vec_j == 0)) or np.any((vec_j == 1) & (vec_i == 0))
        has_ambiguity = np.any((vec_i == 1) & np.isin(vec_j, [-1, 3])) or \
            np.any((vec_j == 1) & np.isin(vec_i, [-1, 3]))

        tier_type = None
        if is_total:
            tier_type = "Total"
        elif has_diff and not has_ambiguity:
            tier_type = "Solid"
        elif has_diff and has_ambiguity:
            tier_type = "Easy"

```

```

        if not tier_type:
            continue # Not a valid mismatch

        # --- Clustering Evaluation ---
        active_indices = np.where(((vec_i == 1) & (vec_j == 0)) | ((vec_i == 0) & (vec_j == 1)))[0]

        if len(active_indices) == 0:
            continue

        active_clusters = set(cluster_mapping[idx] for idx in active_indices)

        cluster_type = "Same" if (len(active_clusters) == 1 and len(active_indices) != 1)
        else "Diff"
        if "Same" == cluster_type:
            tier_type = "All"

        composite_key = (tier_type, cluster_type)

        # --- 3. Composite Quota Verification ---
        if counts[composite_key] < quotas[composite_key]:
            row_data = df_neg.iloc[i].copy()
            row_data["report"] = df_build_neg.loc[j, "report"]
            row_data["section_findings"] = df_build_neg.loc[j, "section_findings"]
            row_data["section_impression"] = df_build_neg.loc[j, "section_impression"]
            row_data["section_summary"] = df_build_neg.loc[j, "section_summary"]

            row_data["mismatch_tier"] = tier_type
            row_data["cluster_tier"] = cluster_type

            df_build_neg = df_build_neg.drop(index=j).reset_index(drop=True)
            mismatch_rows.append(row_data)
            counts[composite_key] += 1
            break

    result_df = pd.DataFrame(mismatch_rows).reset_index(drop=True)
    print("Generation Successful! Sub-distribution counts:")
    for k, v in counts.items():
        print(f" {k}: {v}")
    return result_df

```

EXPLICATION DÉTAILLÉE

- **pure_pathologies** liste les 12 pathologies utilisées pour comparer deux dossiers. Les colonnes *No Finding* et *Support Devices* sont volontairement exclues : elles ne décrivent pas une pathologie à proprement parler.
- **cluster_mapping** regroupe ces 12 pathologies en quatre familles cliniques. Deux pathologies de familles différentes sont considérées plus faciles à distinguer que deux pathologies d'une même famille, qui se ressemblent davantage sur une radiographie.
- **quotas** fixe, pour chaque combinaison (niveau de difficulté, proximité clinique), le nombre d'exemples visés sur les 20 000 à générer. La boucle principale s'arrête dès que la somme de tous les compteurs atteint ce total.

- La boucle externe (**for i in range(...)**) parcourt chaque patient négatif. Pour chacun, elle part d'un point de départ tiré au hasard dans le réservoir (**start_j**), puis explore le réservoir en boucle jusqu'à trouver un candidat compatible avec un quota encore disponible.
- **is_total**, **has_diff** et **has_ambiguity** sont trois tests vectorisés qui comparent les pathologies des deux dossiers pour déterminer le niveau de difficulté : aucune pathologie commune (Total), une divergence nette sans ambiguïté (Solid), ou une divergence avec une pathologie non mentionnée ou incertaine (Easy).
- **active_clusters** identifie les familles cliniques concernées par les pathologies qui diffèrent entre les deux dossiers, ce qui permet de déterminer si l'incohérence se situe entre deux familles différentes (Diff) ou à l'intérieur d'une même famille (Same).
- Lorsqu'un candidat valide est trouvé et que son quota n'est pas saturé, la fonction substitue le compte rendu du candidat (report et ses sous-sections) à la ligne du patient négatif, retire ce candidat du réservoir pour ne jamais le réutiliser, et passe au patient suivant.
- La fonction retourne un dataframe contenant toutes les paires incohérentes générées, et affiche un récapitulatif du nombre d'exemples obtenus pour chaque combinaison de quotas.

À RETENIR - Les trois niveaux de difficulté

Le projet distingue trois niveaux d'incohérence : une incohérence totale, où les deux dossiers ne partagent aucune pathologie (le cas le plus facile à détecter) ; une divergence nette, avec au moins une pathologie clairement contradictoire ; et une divergence ambiguë, où la contradiction s'accompagne d'une pathologie non mentionnée ou incertaine sur l'un des deux dossiers. Au niveau conceptuel, la documentation du projet résume cette répartition par difficulté à 50 % d'incohérence totale, 30 % de divergence nette et 20 % de divergence ambiguë.

À RETENIR - Une seconde dimension : la proximité clinique

Au-delà du niveau de difficulté, le code ajoute une seconde dimension : la proximité clinique des pathologies échangées, via les quatre familles anatomiques. Une incohérence entre deux familles différentes est en général plus facile à repérer qu'une incohérence interne à une même famille, ce qui justifie un quota dédié pour ce cas plus subtil (20 % du total, tous niveaux de difficulté confondus).

À RETENIR - Pourquoi ne pas se contenter d'incohérences grossières ?

Si toutes les paires incohérentes étaient évidentes, le modèle pourrait apprendre un raccourci paresseux plutôt que de comparer véritablement l'image et le texte. Le fait d'inclure aussi des incohérences subtiles, proches cliniquement, rend la tâche plus réaliste et plus exigeante.

Cellule 7 - Exécution de la fabrication des incohérences

CE QUE FAIT CETTE CELLULE

Cette cellule appelle la fonction définie précédemment pour générer effectivement les 20 000 paires incohérentes.

LE CODE

```
# Dataframe containing resulted mismatches
df_result_neg = generate_cluster_balanced_mismatch(df_neg, df_build_neg)
```

EXPLICATION DÉTAILLÉE

- Cet appel peut prendre un certain temps à s'exécuter, puisqu'il peut parcourir une grande partie du réservoir de rapports pour chaque patient négatif.
- La fonction affiche en sortie le nombre d'exemples générés pour chaque combinaison de quotas, ce qui permet de vérifier immédiatement que la génération s'est déroulée comme prévu, avant de poursuivre.

Cellule 8 - Vérification de l'absence de doublons

CE QUE FAIT CETTE CELLULE

Cette cellule vérifie qu'aucun rapport utilisé pour fabriquer les incohérences ne correspond en réalité à un rapport déjà présent parmi les positifs ou parmi les négatifs d'origine.

LE CODE

```
#Checking that we don't have a report from neither the matches dataset sample
# nor the initial mismatches dataset sample
for item in df_result_neg["report"]:
    if (item in df_neg["report"]) or (item in df_pos["report"]):
        print("Not good! Some reports are repeated")
```

EXPLICATION DÉTAILLÉE

- La boucle parcourt chaque rapport utilisé pour les incohérences et vérifie sa présence dans les rapports d'origine des patients négatifs (df_neg) et dans les rapports des patients positifs (df_pos).
- Si aucun message d'alerte ne s'affiche à l'exécution, cela confirme qu'aucune duplication n'a échappé au mécanisme de retrait du réservoir mis en place dans la fonction précédente.

À RETENIR - Pourquoi ce contrôle ?

C'est une vérification d'intégrité classique en ingénierie des données : avant de faire confiance à un jeu de données construit par un algorithme, on vérifie qu'il respecte bien les contraintes voulues - ici, qu'aucun rapport n'a été utilisé deux fois ou associé par erreur à son propre patient.

Cellule 9 - Assemblage du jeu de données final

CE QUE FAIT CETTE CELLULE

Cette cellule réduit positifs et négatifs à 18 000 exemples chacun, attribue l'étiquette cible, puis mélange complètement le jeu de données final.

LE CODE

```
SAMPLE_SIZE = 18000
df_result_neg = df_result_neg.sample(SAMPLE_SIZE, random_state=42)
df_result_neg["target"] = 0
df_pos = df_pos.sample(SAMPLE_SIZE, random_state=42)
df_pos["target"] = 1
dataset_sample = pd.concat([df_pos, df_result_neg], ignore_index=True)
dataset_sample = dataset_sample.sample(frac=1, random_state=42).reset_index(drop=True)
dataset_sample = dataset_sample.iloc[:, :7]
```

EXPLICATION DÉTAILLÉE

- **SAMPLE_SIZE = 18000** fixe la taille finale visée pour chaque classe.
- Les deux échantillons sont réduits de 20 000 à 18 000 exemples chacun ; la marge de 2 000 sert de filet de sécurité, au cas où la fabrication des incohérences n'aurait pas atteint exactement 20 000 candidats valides.
- **target = 0** pour les paires incohérentes, **target = 1** pour les paires cohérentes : c'est l'étiquette binaire que tous les modèles devront ensuite apprendre à prédire.
- **pd.concat(...)** regroupe les deux classes en un seul tableau, puis **sample(frac=1, ...)** mélange entièrement les lignes (frac=1 signifie que 100 % des lignes sont tirées, dans un ordre aléatoire), pour éviter que les exemples positifs et négatifs ne restent groupés par blocs.
- **iloc[:, :7]** ne conserve que les sept premières colonnes utiles (chemin d'image, identifiant patient, rapport et ses sous-sections, et la cible), en écartant les colonnes techniques de pathologies et de niveaux de difficulté qui n'ont servi qu'à la construction.

À RETENIR - Un jeu de données volontairement équilibré

Le choix d'un équilibre strict à 50/50 entre paires cohérentes et incohérentes est délibérément différent de la réalité du terrain, où les erreurs d'association sont rares. Cet équilibre artificiel facilite l'entraînement et l'évaluation d'un classifieur binaire, en évitant les biais liés à des classes déséquilibrées.

Cellule 10 - Aperçu du jeu de données final

CE QUE FAIT CETTE CELLULE

Cette cellule affiche un aperçu du jeu de données assemblé, pour vérification visuelle avant sauvegarde.

LE CODE

```
dataset_sample
```

EXPLICATION DÉTAILLÉE

- Cette ligne, en fin de cellule, affiche automatiquement le dataframe dans la sortie du notebook (comportement standard de Jupyter pour la dernière expression d'une cellule), ce qui permet un contrôle visuel rapide : nombre de lignes, colonnes présentes, répartition apparente des cibles.

Cellule 11 - Sauvegarde du jeu de données

CE QUE FAIT CETTE CELLULE

Cette dernière cellule sauvegarde le jeu de données final sur disque, sous forme de fichier CSV.

LE CODE

```
# Save the dataset sample that will be used in models training into a csv file
dataset_sample.to_csv("./chexpert_plus_dataset_sample.csv")
```

EXPLICATION DÉTAILLÉE

- **to_csv(...)** écrit le dataframe final dans le fichier `chexpert_plus_dataset_sample.csv`.

- Ce fichier est ensuite chargé tel quel par les deux notebooks de modélisation suivants, ce qui garantit que tous les modèles sont entraînés et évalués exactement sur le même jeu de données, avec le même découpage et les mêmes 36 000 paires.

À RETENIR - Pourquoi séparer construction et modélisation ?

Construire le jeu de données est une étape coûteuse en temps de calcul (la fabrication des incohérences parcourt un grand réservoir de rapports). La sauvegarder une fois pour toutes dans un fichier évite de la refaire à chaque expérimentation de modèle, et garantit la reproductibilité des comparaisons entre modèles.

PARTIE B

Modélisation avec BiomedCLIP

Notebook : *models_biomed_clip*. Ce notebook explore quatre approches de complexité croissante avec BiomedCLIP, un modèle pré-entraîné qui associe un Vision Transformer (pour l'image) et PubMedBERT (pour le texte biomédical), entraîné par apprentissage contrastif sur de grandes quantités de paires image-texte du domaine médical. Les quatre approches sont, dans l'ordre : la similarité cosinus entre vecteurs globaux (sans apprentissage), une régression logistique sur un vecteur enrichi, une attention croisée avec MLP de classification, et un ajustement léger du modèle par LoRA.

Cellule 1 - Importation des bibliothèques

CE QUE FAIT CETTE CELLULE

Cette cellule charge l'ensemble des outils nécessaires aux quatre approches du notebook : manipulation de données, chargement de BiomedCLIP, métriques de classification, réseaux de neurones et ajustement léger par LoRA.

LE CODE

```
import pandas as pd
import numpy as np
import gc
import copy

from sklearn.model_selection import train_test_split
import torch
from PIL import Image
from open_clip import create_model_from_pretrained, get_tokenizer, create_model_and_transforms
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score, accuracy_score, precision_score, \
    recall_score, f1_score, confusion_matrix, classification_report
from transformers import AutoModel, AutoTokenizer
from torchvision import transforms
from sklearn.linear_model import LogisticRegression
from torch.utils.data import TensorDataset, DataLoader, Dataset
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import StandardScaler
from peft import LoraConfig, get_peft_model
from torch.amp import autocast, GradScaler
from torchinfo import summary
```

EXPLICATION DÉTAILLÉE

- **gc** et **copy** servent respectivement à libérer explicitement la mémoire (garbage collector) et à dupliquer en profondeur les poids d'un modèle (utile pour conserver la meilleure version d'un réseau pendant l'entraînement).
- **open_clip** est la bibliothèque qui sait charger BiomedCLIP, un modèle construit sur l'architecture CLIP (Contrastive Language-Image Pretraining) mais ré-entraîné sur des données biomédicales.
- **sklearn.metrics** regroupe toutes les fonctions de mesure de performance utilisées dans ce notebook : aire sous la courbe ROC, exactitude, précision, rappel, F1, matrice de confusion et rapport de classification

complet.

- **peft** (Parameter-Efficient Fine-Tuning) fournit LoraConfig et get_peft_model, les outils qui permettront, dans la dernière approche du notebook, d'ajuster BiomedCLIP sans réentraîner l'intégralité de ses paramètres.
- **torch.amp** (Automatic Mixed Precision) fournit autocast et GradScaler, des outils qui accélèrent l'entraînement et réduisent la mémoire utilisée sur carte graphique, en calculant certaines opérations en précision réduite.
- **torchinfo** fournit summary, qui affiche un résumé lisible de l'architecture d'un réseau de neurones et de son nombre de paramètres.

Cellule 2 - Chargement du jeu de données préparé

CE QUE FAIT CETTE CELLULE

Cette cellule charge le fichier CSV construit dans la partie A : les 36 000 paires image-rapport, équilibrées entre cohérentes et incohérentes.

LE CODE

```
pd.options.display.max_columns = None
dataset_sample = pd.read_csv("./chexpert_plus_dataset_sample.csv", index_col=0)
dataset_sample
```

EXPLICATION DÉTAILLÉE

- **index_col=0** indique que la première colonne du fichier CSV doit être utilisée comme index du dataframe plutôt que comme une colonne de données ordinaire (cette colonne correspond à l'ancien index pandas, sauvegardé automatiquement par to_csv dans la partie A).
- Le chargement de ce fichier unique garantit que ce notebook, et le suivant, travaillent tous deux sur exactement les mêmes 36 000 paires.

Cellule 3 - Découpage en jeu d'entraînement et de test

CE QUE FAIT CETTE CELLULE

Cette cellule sépare les données en un jeu d'entraînement (80 %) et un jeu de test (20 %), en conservant la même proportion de paires cohérentes et incohérentes dans les deux.

LE CODE

```
X = dataset_sample.drop(columns=["target"])
y = dataset_sample["target"]

# Split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

EXPLICATION DÉTAILLÉE

- **X** regroupe toutes les colonnes sauf la cible (target) ; **y** est la colonne cible seule.
- **test_size=0.2** réserve 20 % des 36 000 paires pour le test, soit 7 200 exemples, les 80 % restants (28 800 exemples) servant à l'entraînement.
- **stratify=y** garantit que la proportion de paires cohérentes et incohérentes est identique dans le jeu d'entraînement et dans le jeu de test, ce qui évite qu'un découpage malchanceux ne déséquilibre artificiellement l'une des deux classes.

À RETENIR - Pourquoi stratifier ?

Sans stratification, un tirage aléatoire pourrait par hasard produire un jeu de test avec, par exemple, 60 % de paires cohérentes au lieu de 50 %. L'évaluation du modèle serait alors moins représentative de la réalité du jeu de données complet.

Cellule 4 - Chargement du modèle BiomedCLIP

CE QUE FAIT CETTE CELLULE

Cette cellule charge le modèle BiomedCLIP pré-entraîné ainsi que son tokenizer, et le place en mode évaluation sur la carte graphique si elle est disponible.

LE CODE

```
# Load the BiomedCLIP model and processor
model, preprocess = create_model_from_pretrained(
    'hf-hub:microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224')
tokenizer = get_tokenizer(
    'hf-hub:microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224')

device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model.to(device)
model.eval()
```

EXPLICATION DÉTAILLÉE

- **create_model_from_pretrained(...)** télécharge et instancie BiomedCLIP depuis le Hub Hugging Face, et retourne à la fois le modèle et la fonction de prétraitement d'image (redimensionnement, normalisation) attendue par ce modèle.
- **get_tokenizer(...)** retourne le tokenizer associé, qui transforme un texte en suite de nombres compréhensible par PubMedBERT.
- **device** sélectionne automatiquement la carte graphique (cuda) si elle est disponible, ou le processeur sinon ; **model.to(device)** y transfère les poids du modèle.
- **model.eval()** place le modèle en mode évaluation : certains mécanismes propres à l'entraînement (comme le dropout, qui désactive aléatoirement des neurones) sont coupés, pour obtenir des résultats stables et reproductibles.

À RETENIR - BiomedCLIP en bref

BiomedCLIP associe un Vision Transformer (qui découpe une image en petites régions, ou patches, et en extrait une représentation) et PubMedBERT (un modèle de langage entraîné sur la littérature biomédicale). Les deux encodeurs ont été pré-entraînés ensemble par apprentissage contrastif : ils apprennent à rapprocher les vecteurs d'une image et de son texte associé, et à éloigner ceux de paires non associées.

Cellule 5 - Première approche : fonction de similarité cosinus

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction au cœur de la première approche : elle encode une image et un texte séparément, puis mesure leur proximité par similarité cosinus.

LE CODE

```
# Extract the encoded image/text vectors and compute the similarity score between them
def check_biomed_clip_match(image_path, report_text):
    # Prepare inputs
    image = preprocess(Image.open(image_path)).unsqueeze(0).to(device)
    text = tokenizer([report_text]).to(device)

    with torch.no_grad():
        # Get features
        image_features = model.encode_image(image)
        text_features = model.encode_text(text)

        # Normalize features
        image_features /= image_features.norm(dim=-1, keepdim=True)
        text_features /= text_features.norm(dim=-1, keepdim=True)

        # Calculate similarity score (ranges roughly from -1 to 1, usually >0)
        similarity = (image_features @ text_features.T).item()

    return similarity
```

EXPLICATION DÉTAILLÉE

- **preprocess(...)** applique à l'image les transformations attendues par BiomedCLIP (redimensionnement, normalisation), et **unsqueeze(0)** ajoute une dimension de lot (batch) de taille 1, car le modèle attend toujours un lot d'images, même réduit à une seule.
- **torch.no_grad()** désactive le calcul des gradients : puisqu'on ne fait ici que de l'inférence (pas d'entraînement), cela économise mémoire et temps de calcul.
- **encode_image** et **encode_text** font passer l'image et le texte dans leurs encodeurs respectifs et retournent chacun un vecteur numérique de même dimension.
- La **normalisation** (division par la norme) ramène chaque vecteur à une longueur de 1, ce qui permet d'interpréter le produit scalaire qui suit comme une similarité cosinus pure, indépendante de la longueur des vecteurs.
- **image_features @ text_features.T** calcule le produit scalaire entre les deux vecteurs normalisés : c'est la similarité cosinus, un nombre généralement compris entre -1 et 1, d'autant plus élevé que l'image et le texte sont jugés proches par le modèle.

À RETENIR - La similarité cosinus

La similarité cosinus mesure l'angle entre deux vecteurs plutôt que leur distance brute : deux vecteurs pointant dans la même direction ont une similarité proche de 1, deux vecteurs perpendiculaires une similarité proche de 0. C'est une mesure très utilisée pour comparer des représentations apprises par des réseaux de neurones.

Cellule 6 - Calcul du seuil optimal sur le jeu d'entraînement

CE QUE FAIT CETTE CELLULE

Cette cellule calcule la similarité de chaque paire du jeu d'entraînement, puis détermine le seuil de similarité qui sépare le mieux les paires cohérentes des incohérentes.

LE CODE

```
# Get the similarity scores on the train set
X_train["similarity_score"] = 0.0
for i in X_train.index:
    image_path = "./CheXpert/CheXpert-v1.0-small/" + X_train.loc[i, "path_to_image"]
    report_text = X_train.loc[i, "report"]
    X_train.loc[i, "similarity_score"] = check_biomed_clip_match(image_path, report_text)

# Function returning the similarity score threshold
def evaluate(targets, similarities):
    # Compute ROC-AUC Score
    auc = roc_auc_score(targets, similarities)
    print(f"Test ROC-AUC Score: {auc:.4f}")

    # Calculate ROC Curve metrics
    fpr, tpr, thresholds = roc_curve(targets, similarities)

    # Find optimal threshold
    j_scores = tpr - fpr
    best_idx = np.argmax(j_scores)
    best_threshold = thresholds[best_idx]

    print(f"Optimal Similarity Threshold: {best_threshold:.4f}")
    print(f"Maximized True Positive Rate (Sensitivity): {tpr[best_idx]:.4f}")
    print(f"Minimized False Positive Rate (1 - Specificity): {fpr[best_idx]:.4f}")

    # Plot the ROC Curve
    plt.figure(figsize=(6, 5))
    plt.plot(fpr, tpr, label=f'BiomedCLIP Baseline (AUC = {auc:.2f})', color='darkorange', lw=2)
    plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
    plt.scatter(fpr[best_idx], tpr[best_idx], color='red',
                label=f'Best Threshold ({best_threshold:.3f})', zorder=5)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Receiver Operating Characteristic (ROC) Curve')
```

```
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.show()

return best_threshold

targets = y_train.to_numpy()
similarities = X_train["similarity_score"].to_numpy()
# Compute the similarity score threshold on the train set
opt_threshold = evaluate(targets, similarities)
```

EXPLICATION DÉTAILLÉE

- La boucle calcule la similarité cosinus pour chacune des 28 800 paires d'entraînement, et stocke le résultat dans une nouvelle colonne `similarity_score`.
- **roc_auc_score** calcule l'aire sous la courbe ROC (Receiver Operating Characteristic), une mesure globale de la capacité du score de similarité à distinguer les deux classes, indépendamment de tout seuil de décision.
- **roc_curve** calcule, pour une multitude de seuils possibles, le taux de vrais positifs (tpr) et le taux de faux positifs (fpr) associés.
- Le **J de Youden** ($j_scores = tpr - fpr$) est un critère classique pour choisir un seuil de décision : il identifie le point de la courbe ROC qui maximise l'écart entre vrais positifs et faux positifs, c'est-à-dire le meilleur compromis entre sensibilité et spécificité.
- Le seuil optimal est calculé uniquement sur le **jeu d'entraînement**, puis réutilisé tel quel sur le jeu de test dans la cellule suivante - une précaution essentielle pour ne pas biaiser l'évaluation.

À RETENIR - Résultat obtenu

Sur le jeu d'entraînement, cette première approche atteint un ROC-AUC d'environ 0,66, avec un seuil optimal proche de 0,39. C'est un résultat modeste : la simple similarité entre vecteurs globaux peine à séparer nettement les paires cohérentes des incohérentes, ce qui motive les approches suivantes, plus sophistiquées.

Cellule 7 - Évaluation complète de la similarité cosinus

CE QUE FAIT CETTE CELLULE

Cette cellule calcule l'ensemble des métriques de classification (exactitude, précision, rappel, F1, spécificité) pour la première approche, sur le jeu d'entraînement puis sur le jeu de test.

LE CODE

```
# Display the metrics of the binary classification
def bin_classif_metrics(similarities, targets, opt_threshold, type='Train'):
    # Convert continuous similarity scores into binary predictions (0 or 1)
    predictions = (similarities >= opt_threshold).astype(int)

    # Calculate operational metrics
    accuracy = accuracy_score(targets, predictions)
    precision = precision_score(targets, predictions)
    recall = recall_score(targets, predictions)
```

```

f1 = f1_score(targets, predictions)

# Extract Specificity (True Negative Rate) via Confusion Matrix
tn, fp, fn, tp = confusion_matrix(targets, predictions).ravel()
specificity = tn / (tn + fp)

# Print the evaluation dashboard
print(f"--- Operational Metrics at Threshold {opt_threshold} ---")
print(f"{type} ROC-AUC Score: {roc_auc_score(targets, similarities):.4f}")
print(f"{type} Accuracy: {accuracy:.4f}")
print(f"{type} Precision: {precision:.4f}")
print(f"{type} Recall: {recall:.4f}")
print(f"{type} F1-Score: {f1:.4f}")
print(f"{type} Specificity: {specificity:.4f} (FPR is {1-specificity:.4f})")

print("-----Train metrics-----")
targets = y_train.to_numpy()
similarities = X_train["similarity_score"].to_numpy()
bin_classif_metrics(similarities, targets, opt_threshold, type='Train')

print("-----Test metrics-----")
# Get the similarity scores on the train set
X_test["similarity_score"] = 0.0
for i in X_test.index:
    image_path = "./CheXpert/CheXpert-v1.0-small/" + X_test.loc[i, "path_to_image"]
    report_text = X_test.loc[i, "report"]
    X_test.loc[i, "similarity_score"] = check_biomed_clip_match(image_path, report_text)
targets = y_test.to_numpy()
similarities = X_test["similarity_score"].to_numpy()
bin_classif_metrics(similarities, targets, opt_threshold, type='Test')

```

EXPLICATION DÉTAILLÉE

- **predictions = (similarities >= opt_threshold)** transforme le score continu de similarité en décision binaire (0 ou 1), en appliquant le seuil calculé à la cellule précédente.
- **accuracy_score**, **precision_score**, **recall_score** et **f1_score** sont les quatre métriques de classification standard, calculées en comparant les prédictions aux vraies étiquettes.
- La **spécificité** (taux de vrais négatifs) n'a pas de fonction dédiée dans scikit-learn et se calcule donc manuellement à partir de la matrice de confusion : $tn / (tn + fp)$.
- La cellule applique cette évaluation deux fois : d'abord sur le jeu d'entraînement (avec les scores déjà calculés), puis sur le jeu de test, pour lequel les similarités sont calculées ici pour la première fois.

À RETENIR - Résultat obtenu

Sur le jeu de test, cette première approche atteint une exactitude d'environ 0,62, une précision de 0,64, un rappel de 0,55 et un F1 de 0,59 - des résultats du même ordre que sur l'entraînement, ce qui montre que le modèle ne sur-apprend pas, mais confirme aussi les limites d'une approche fondée sur un unique vecteur global par image et par texte.

Cellule 8 - Deuxième approche : extraction d'un vecteur enrichi

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction utilisée par la deuxième approche : au lieu de calculer une simple similarité, elle construit un vecteur enrichi combinant l'image, le texte, leur différence et leur produit terme à terme.

LE CODE

```
# Just getting the encoded vectors image/text
def extract_multimodal_features(image_path, report_text):
    image = preprocess(Image.open(image_path)).unsqueeze(0).to(device)
    text = tokenizer([report_text]).to(device)

    with torch.no_grad():
        image_features = model.encode_image(image)
        text_features = model.encode_text(text)

    # Normalize the vectors
    image_features /= image_features.norm(dim=-1, keepdim=True)
    text_features /= text_features.norm(dim=-1, keepdim=True)

    # Move to CPU and convert to numpy for scikit-learn
    img_emb = image_features.cpu().numpy().flatten()
    txt_emb = text_features.cpu().numpy().flatten()

    # Combine features: Absolute difference and Element-wise product
    abs_diff = np.abs(img_emb - txt_emb)
    product = img_emb * txt_emb

    # Concatenate them to create a single robust feature vector for this pair
    combined_feature = np.concatenate([img_emb, txt_emb, abs_diff, product])
    return combined_feature
```

EXPLICATION DÉTAILLÉE

- Les premières lignes sont identiques à la fonction de la cellule 5 : encodage de l'image et du texte, puis normalisation des deux vecteurs.
- **img_emb** et **txt_emb** sont ramenés sur le processeur (cpu) et convertis en tableaux numpy à une dimension (flatten), pour être utilisables par scikit-learn.
- **abs_diff** (différence absolue terme à terme) et **product** (produit terme à terme) sont deux manières classiques d'expliciter la relation entre deux vecteurs : la différence souligne les écarts, le produit souligne les similitudes.
- **np.concatenate(...)** assemble les quatre vecteurs (image, texte, différence, produit) en un seul grand vecteur, qui contient ainsi à la fois l'information brute des deux modalités et des indices explicites de leur relation.

À RETENIR - Pourquoi enrichir le vecteur ?

La similarité cosinus (première approche) réduit toute la comparaison à un seul nombre. En conservant les vecteurs complets, ainsi que leur différence et leur produit, on donne beaucoup plus d'information à un modèle de classification, qui peut alors apprendre lui-même quelles combinaisons de dimensions sont les plus utiles pour distinguer les paires cohérentes des incohérentes.

Cellule 9 - Construction des vecteurs enrichis pour l'entraînement et le test

CE QUE FAIT CETTE CELLULE

Cette cellule applique la fonction précédente à toutes les paires d'entraînement et de test, pour construire les jeux de données qui alimenteront la régression logistique.

LE CODE

```
# Extract multimodal features for train and test sets:
reg_X_train = np.array([extract_multimodal_features(img, txt) for img, txt in zip(
    ("./CheXpert/CheXpert-v1.0-small/" + X_train["path_to_image"]).tolist(),
    X_train["report"].tolist())])
reg_y_train = y_train.to_numpy()

reg_X_test = np.array([extract_multimodal_features(img, txt) for img, txt in zip(
    ("./CheXpert/CheXpert-v1.0-small/" + X_test["path_to_image"]).tolist(),
    X_test["report"].tolist())])
reg_y_test = y_test.to_numpy()
```

EXPLICATION DÉTAILLÉE

- Chaque ligne est une compréhension de liste qui appelle `extract_multimodal_features` pour chaque paire (chemin d'image, rapport), en parcourant en parallèle les deux colonnes grâce à **`zip(...)`**.
- **`np.array(...)`** rassemble tous les vecteurs obtenus en une seule matrice numpy, prête à être utilisée par un modèle scikit-learn.
- Cette étape est répétée pour le jeu d'entraînement et pour le jeu de test, en réutilisant à chaque fois le même modèle BiomedCLIP gelé (non réentraîné) pour encoder les images et les textes.

Cellule 10 - Entraînement de la régression logistique

CE QUE FAIT CETTE CELLULE

Cette cellule entraîne un modèle de régression logistique sur les vecteurs enrichis du jeu d'entraînement.

LE CODE

```
# Train Logistic Regression
log_reg = LogisticRegression(max_iter=5000, random_state=42)
log_reg.fit(reg_X_train, reg_y_train)
```

EXPLICATION DÉTAILLÉE

- **`LogisticRegression(max_iter=5000, ...)`** instancie un modèle de régression logistique scikit-learn ; `max_iter` relève la limite par défaut du nombre d'itérations d'optimisation, nécessaire ici car le vecteur d'entrée est de grande dimension (la concaténation de quatre vecteurs).
- **`fit(...)`** entraîne le modèle : il cherche les coefficients qui séparent au mieux les vecteurs des paires cohérentes de ceux des paires incohérentes.

À RETENIR - Une régression logistique, malgré la complexité des vecteurs

Même si les vecteurs d'entrée proviennent d'un grand modèle de deep learning, le classifieur final reste ici un modèle simple et interprétable. C'est une approche hybride courante : utiliser un grand modèle pré-entraîné comme extracteur de caractéristiques, puis un modèle plus léger pour la décision finale.

Cellule 11 - Évaluation de la régression logistique

CE QUE FAIT CETTE CELLULE

Cette cellule évalue le modèle entraîné sur le jeu d'entraînement puis sur le jeu de test, et affiche un rapport de classification complet pour chacun.

LE CODE

```
# Predict probabilities on the train and test sets
y_probs_train = log_reg.predict_proba(reg_X_train)[: , 1]
y_preds_train = log_reg.predict(reg_X_train)
y_probs_test = log_reg.predict_proba(reg_X_test)[: , 1]
y_preds_test = log_reg.predict(reg_X_test)

# Evaluate
print("-----Train metrics-----")
print(f"Train ROC-AUC Score: {roc_auc_score(reg_y_train, y_probs_train):.4f}")
print("Train Classification Report:")
print(classification_report(reg_y_train, y_preds_train))
print("-----Test metrics-----")
print(f"Test ROC-AUC Score: {roc_auc_score(reg_y_test, y_probs_test):.4f}")
print("Test Classification Report:")
print(classification_report(reg_y_test, y_preds_test))
```

EXPLICATION DÉTAILLÉE

- **predict_proba(...)[, 1]** retourne la probabilité prédite que la paire soit cohérente (classe 1) ; **predict(...)** retourne directement la classe prédite (0 ou 1), au seuil par défaut de 0,5.
- **classification_report(...)** est une fonction scikit-learn qui affiche en une seule fois la précision, le rappel et le F1 pour chaque classe, ainsi que les moyennes globales.

À RETENIR - Résultat obtenu

La régression logistique sur vecteurs enrichis atteint un ROC-AUC d'environ 0,83 sur le test (contre 0,66 pour la similarité cosinus), avec une exactitude d'environ 0,75 et un F1 moyen d'environ 0,75. C'est une nette amélioration par rapport à la première approche : conserver l'information complète des vecteurs, plutôt que de la réduire à un seul score, aide considérablement le modèle.

Cellule 12 - Troisième approche : réduction de l'échantillon pour l'attention croisée

CE QUE FAIT CETTE CELLULE

Cette cellule réduit la taille du jeu de données utilisé pour l'attention croisée, afin de limiter le temps de calcul et le risque de saturation mémoire, puis effectue un nouveau découpage entraînement / test.

LE CODE

```
df_pos = dataset_sample.loc[dataset_sample["target"] == 1, :]
df_neg = dataset_sample.loc[dataset_sample["target"] == 0, :]

df_pos = df_pos.sample(11000, random_state=42).reset_index(drop=True)
df_neg = df_neg.sample(11000, random_state=42).reset_index(drop=True)

df_concat = pd.concat([df_pos, df_neg], ignore_index=True)
df_concat = df_concat.sample(frac=1, random_state=42).reset_index(drop=True)

X_cross_att = df_concat.drop(columns=["target"])
y_cross_att = df_concat["target"]

# Split with stratification
X_train_cross_att, X_test_cross_att, y_train_cross_att, y_test_cross_att = train_test_split(
    X_cross_att,
    y_cross_att,
    test_size=0.2,
    random_state=42,
    stratify=y_cross_att,
)
```

EXPLICATION DÉTAILLÉE

- Le jeu complet (36 000 paires) est réduit à 22 000 paires (11 000 cohérentes et 11 000 incohérentes), puis mélangé et redécoupé en entraînement (80 %, soit 17 600 paires) et test (20 %, soit 4 400 paires).
- Cette réduction est justifiée dans le notebook par un constat pratique : l'attention croisée manipule des tenseurs bien plus volumineux que les approches précédentes (une matrice de patches d'image et une séquence de tokens de texte par exemple, au lieu d'un seul vecteur), ce qui rend l'entraînement sur les 36 000 paires complètes trop long et risqué en mémoire, pour un gain de performance jugé négligeable.

À RETENIR - Un compromis assumé et documenté

Réduire la taille d'un jeu de données est un choix d'ingénierie courant lorsque les ressources de calcul sont limitées. L'important est, comme ici, de le documenter explicitement et de vérifier que les résultats restent comparables.

Cellule 13 - Fonction d'extraction des représentations détaillées**CE QUE FAIT CETTE CELLULE**

Cette cellule définit une nouvelle fonction d'encodage, qui ne retourne plus un seul vecteur global par image et par texte, mais l'ensemble des représentations intermédiaires nécessaires à l'attention croisée.

LE CODE

```
def biomed_clip_cross_att(image_path, report_text):  
    # Prepare inputs  
    image = preprocess(Image.open(image_path)).unsqueeze(0).to(device)  
    text = tokenizer([report_text]).to(device)  
  
    with torch.no_grad():  
        # Extract Image Patches Embeddings  
        # Output shape: [Batch, Num_Patches, Vision_Dim] (e.g., [B, 197, 768])  
        # 197 includes 196 patches (14x14) + 1 CLS token  
        img_patches = model.visual.trunk.forward_features(image)  
        img_patches = img_patches[:, 1:, :]  
  
        # Extracts sequence embeddings from PubMedBERT before pooler squashing.  
        text_outputs = model.text.transformer(text)  
        text_tokens = text_outputs.last_hidden_state  
  
        img_patches_np = img_patches.squeeze(0).cpu().numpy()  
        text_tokens_np = text_tokens.squeeze(0).cpu().numpy()  
  
    return img_patches_np, text_tokens_np
```

EXPLICATION DÉTAILLÉE

- Contrairement aux fonctions précédentes, celle-ci n'appelle pas `encode_image` ni `encode_text` (qui retournent un vecteur unique résumant toute l'image ou tout le texte), mais accède directement aux couches internes du modèle.
- **`model.visual.trunk.forward_features(image)`** retourne, pour chaque image, la représentation de chacune de ses 196 régions (patches) de 16x16 pixels, plus un jeton de synthèse (CLS) ; `[:, 1:, :]` retire ce jeton CLS pour ne garder que les 196 patches.
- **`model.text.transformer(text)`** retourne, pour chaque mot (ou sous-mot) du rapport, sa représentation contextuelle, avant que ces informations ne soient résumées en un seul vecteur.
- Le résultat est donc, pour chaque paire, une grille de représentations d'image et une séquence de représentations de texte - exactement ce dont l'attention croisée a besoin pour faire dialoguer chaque mot avec chaque région de l'image.

À RETENIR - Pourquoi ne plus résumer en un seul vecteur ?

Réduire une image ou un texte à un seul vecteur global, comme dans les deux premières approches, fait perdre l'information sur la localisation : on sait que l'image et le texte se ressemblent globalement, mais pas quelle région de l'image correspond à quel passage du texte. Conserver les représentations détaillées permet à l'attention croisée d'apprendre ces correspondances fines.

Cellule 14 - Extraction des représentations détaillées sur le jeu d'entraînement

CE QUE FAIT CETTE CELLULE

Cette cellule applique la fonction précédente à toutes les paires du jeu d'entraînement réduit, et assemble les résultats en tableaux numpy.

LE CODE

```
cross_att_X_train_txt = []
cross_att_X_train_img = []
for i, (img, txt) in enumerate(zip(
    ("./CheXpert/CheXpert-v1.0-small/" + X_train_cross_att["path_to_image"]).tolist(),
    X_train_cross_att["report"].tolist())):
    img_pat, txt_pat = biomed_clip_cross_att(img, txt)
    cross_att_X_train_img.append(img_pat)
    cross_att_X_train_txt.append(txt_pat)

    if i % 200 == 0 and i > 0:
        gc.collect()
        torch.cuda.empty_cache()

cross_att_y_train = y_train_cross_att.to_numpy()
cross_att_X_train_txt = np.array(cross_att_X_train_txt)
cross_att_X_train_img = np.array(cross_att_X_train_img)
```

EXPLICATION DÉTAILLÉE

- La boucle appelle `biomed_clip_cross_att` pour chacune des 17 600 paires d'entraînement, et accumule les grilles de patches d'image et les séquences de tokens de texte dans deux listes.
- `gc.collect()` et `torch.cuda.empty_cache()`, appelés tous les 200 exemples, forcent la libération de la mémoire graphique inutilisée : comme chaque exemple produit ici des tableaux volumineux (une grille complète de patches, une séquence complète de tokens), un nettoyage périodique évite la saturation progressive de la mémoire.
- Une fois la boucle terminée, les listes Python sont converties en tableaux numpy, plus efficaces pour les étapes suivantes.

Cellule 15 - Extraction des représentations détaillées sur le jeu de test

CE QUE FAIT CETTE CELLULE

Cette cellule reproduit exactement la même extraction que la cellule précédente, sur les 4 400 paires du jeu de test.

LE CODE

```
cross_att_X_test_txt = []
cross_att_X_test_img = []
for i, (img, txt) in enumerate(zip(
    ("./CheXpert/CheXpert-v1.0-small/" + X_test_cross_att["path_to_image"]).tolist(),
    X_test_cross_att["report"].tolist())):
    img_pat, txt_pat = biomed_clip_cross_att(img, txt)
    cross_att_X_test_img.append(img_pat)
    cross_att_X_test_txt.append(txt_pat)

    if i % 200 == 0 and i > 0:
        gc.collect()
        torch.cuda.empty_cache()

cross_att_y_test = y_test_cross_att.to_numpy()
cross_att_X_test_txt = np.array(cross_att_X_test_txt)
```

```
cross_att_X_test_img = np.array(cross_att_X_test_img)
```

EXPLICATION DÉTAILLÉE

- La logique est identique à la cellule 14 : même fonction d'extraction, même nettoyage périodique de la mémoire, mais appliquée cette fois au jeu de test plutôt qu'au jeu d'entraînement.
- Garder cette extraction strictement séparée entre entraînement et test est essentiel : aucune image ni aucun texte du jeu de test ne doit influencer l'entraînement du modèle qui suit.

Cellule 16 - Préparation des tenseurs et des lots d'entraînement

CE QUE FAIT CETTE CELLULE

Cette cellule convertit les tableaux numpy en tenseurs PyTorch, les regroupe en jeux de données, puis prépare les chargeurs qui fourniront les données par lots pendant l'entraînement.

LE CODE

```
cross_att_X_train_txt_tensor = torch.from_numpy(cross_att_X_train_txt).float()
cross_att_X_train_img_tensor = torch.from_numpy(cross_att_X_train_img).float()
cross_att_y_train_tensor = torch.from_numpy(cross_att_y_train).float()
cross_att_train_dataset = TensorDataset(
    cross_att_X_train_img_tensor, cross_att_X_train_txt_tensor, cross_att_y_train_tensor)
cross_att_X_test_txt_tensor = torch.from_numpy(cross_att_X_test_txt).float()
cross_att_X_test_img_tensor = torch.from_numpy(cross_att_X_test_img).float()
cross_att_y_test_tensor = torch.from_numpy(cross_att_y_test).float()
cross_att_test_dataset = TensorDataset(
    cross_att_X_test_img_tensor, cross_att_X_test_txt_tensor, cross_att_y_test_tensor)

cross_att_train_loader = DataLoader(cross_att_train_dataset, batch_size=64, shuffle=True)
cross_att_val_loader = DataLoader(cross_att_test_dataset, batch_size=64, shuffle=False)
```

EXPLICATION DÉTAILLÉE

- **torch.from_numpy(...).float()** convertit chaque tableau numpy en tenseur PyTorch, dans le format numérique attendu par le réseau de neurones.
- **TensorDataset(...)** regroupe les images, les textes et les étiquettes correspondantes en un seul objet, qui garantit que les trois éléments d'une même paire restent toujours associés.
- **DataLoader(..., batch_size=64, shuffle=True)** découpe le jeu d'entraînement en lots (batches) de 64 exemples, mélangés à chaque epoch, pour un entraînement par descente de gradient stochastique par mini-lots.
- Pour le jeu de test (val_loader), **shuffle=False** : l'ordre n'a pas d'importance pour l'évaluation, et le garder fixe facilite le débogage.

Cellule 17 - Architecture du modèle d'attention croisée

CE QUE FAIT CETTE CELLULE

Cette cellule définit l'architecture du réseau de neurones qui fait dialoguer chaque mot du rapport avec chaque région de l'image, puis prend une décision de classification.

LE CODE

```

# Cross-Attention+MLP Model
class CrossAttentionClassifier(nn.Module):
    def __init__(self, embed_dim=768, num_heads=8, dropout=0.3):
        super().__init__()

        self.cross_attention = nn.MultiheadAttention(
            embed_dim=embed_dim,
            num_heads=num_heads,
            dropout=dropout,
            batch_first=True
        )

        self.layer_norm1 = nn.LayerNorm(embed_dim)
        self.layer_norm2 = nn.LayerNorm(embed_dim)

        # Add a small Feed-Forward Network layer (FFN) inside the attention block
        # This mirrors a true standard Transformer block and stabilizes representation warping
        self.ffn = nn.Sequential(
            nn.Linear(embed_dim, embed_dim * 2),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(embed_dim * 2, embed_dim)
        )

        self.classifier = nn.Sequential(
            nn.Linear(embed_dim, 256),
            nn.GELU(),
            nn.LayerNorm(256),
            nn.Dropout(dropout),
            nn.Linear(256, 1)
        )

    def forward(self, img_patches, text_tokens):
        norm_text = self.layer_norm1(text_tokens)
        # Cross-Attention Core
        attn_output, _ = self.cross_attention(
            query=norm_text,
            key=img_patches,
            value=img_patches
        )
        x = (attn_output + text_tokens)

        # FFN stabilization step
        x = self.layer_norm2(self.ffn(x)) + x

        # Max-pooling across sequence dimension
        x_pooled, _ = torch.max(x, dim=1)
        # Predict
        return self.classifier(x_pooled)

```

EXPLICATION DÉTAILLÉE

- **nn.MultiheadAttention** est le mécanisme central : pour chaque mot du texte (la requête, query), il calcule à quel point chacune des 196 régions de l'image (les clés et valeurs, key et value) lui est pertinente, et

construit une représentation du mot enrichie par l'information visuelle la plus utile.

- **num_heads=8** répète ce mécanisme huit fois en parallèle, chaque tête pouvant apprendre à se concentrer sur un type de correspondance différent entre texte et image.
- Les couches **LayerNorm** stabilisent les valeurs numériques entre les différentes étapes du réseau, une pratique standard dans les architectures de type Transformer.
- Le bloc **ffn** (feed-forward network) est une petite couche supplémentaire après l'attention, qui permet au modèle de retraiter l'information avant de la transmettre plus loin - c'est la structure classique d'un bloc Transformer complet.
- **torch.max(x, dim=1)** résume la séquence de mots enrichis en un seul vecteur, en conservant pour chaque dimension la valeur la plus forte sur l'ensemble du texte : cette opération de max-pooling met en avant les signaux les plus marquants, qu'ils viennent d'un mot ou d'un autre.
- **self.classifier** est un petit réseau (perceptron multicouche, MLP) qui transforme ce vecteur résumé en une unique valeur de sortie, utilisée ensuite pour décider cohérent ou incohérent.

À RETENIR - L'attention croisée en une phrase

Au lieu de comparer un seul résumé d'image à un seul résumé de texte, l'attention croisée laisse chaque mot du rapport interroger l'ensemble des régions de l'image et retenir celles qui le concernent le plus, avant de prendre une décision globale.

Cellule 18 - Fonction d'entraînement avec arrêt anticipé

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction d'entraînement du modèle d'attention croisée, avec une boucle classique d'entraînement et de validation, et un mécanisme d'arrêt anticipé qui conserve les meilleurs poids rencontrés.

LE CODE

```
def cross_attention_train(model, train_loader, val_loader, criterion, optimizer, scheduler,
                          epochs=100, patience=3):
    history = {'loss': [], 'val_loss': [], 'accuracy': [], 'val_accuracy': []}

    # Early stopping trackers
    best_val_loss = float('inf')
    patience_counter = 0
    best_model_weights = None
    best_epoch = 0

    for epoch in range(epochs):
        model.train()
        total_loss, correct = 0, 0

        for inputs_img, inputs_txt, labels in train_loader:
            inputs_img = inputs_img.to(device)
            inputs_txt = inputs_txt.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs_img, inputs_txt).squeeze(1)
```

```
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        correct += ((torch.sigmoid(outputs) >= 0.5).float() == labels).sum().item()

train_loss = total_loss / len(train_loader)
train_acc = correct / len(train_loader.dataset)

model.eval()
val_loss, val_correct = 0, 0
with torch.no_grad():
    for inputs_img, inputs_txt, labels in val_loader:
        inputs_img = inputs_img.to(device)
        inputs_txt = inputs_txt.to(device)
        labels = labels.to(device)
        outputs = model(inputs_img, inputs_txt).squeeze(1)
        loss = criterion(outputs, labels)
        val_loss += loss.item()
        val_correct += ((torch.sigmoid(outputs) >= 0.5).float() == labels).sum().item()

val_loss /= len(val_loader)
val_acc = val_correct / len(val_loader.dataset)

scheduler.step()
current_lr = optimizer.param_groups[0]['lr']

history['loss'].append(train_loss)
history['val_loss'].append(val_loss)
history['accuracy'].append(train_acc)
history['val_accuracy'].append(val_acc)

print(f"Epoch [{epoch+1}/{epochs}], Loss: {train_loss:.4f}, Acc: {train_acc:.4f}, "
      f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}, LR: {current_lr:.6f}")

# --- Early Stopping Logic ---
if val_loss < best_val_loss:
    best_val_loss = val_loss
    patience_counter = 0
    best_epoch = epoch + 1
    best_model_weights = copy.deepcopy(model.state_dict())
else:
    patience_counter += 1

if patience_counter >= patience:
    print(f"Early stopping triggered! Stopping training at epoch {epoch+1}.")
    break

if best_model_weights is not None:
    print(f"Restoring best model weights found at Epoch {best_epoch}")
    model.load_state_dict(best_model_weights)
```

```
return history
```

EXPLICATION DÉTAILLÉE

- Pour chaque epoch, la boucle d'entraînement (**model.train()**) parcourt tous les lots du jeu d'entraînement : calcul de la prédiction, calcul de la perte (criterion), rétropropagation du gradient (loss.backward()) et mise à jour des poids (optimizer.step()).
- La boucle de validation (**model.eval()**, sous torch.no_grad()) mesure ensuite la performance du modèle sur le jeu de test, sans jamais mettre à jour ses poids à partir de ces données.
- **scheduler.step()** fait évoluer le taux d'apprentissage au fil des epochs, ici selon un calendrier de décroissance progressive (défini à la cellule suivante).
- Le mécanisme d'**arrêt anticipé** (early stopping) surveille la perte de validation à chaque epoch : si elle s'améliore, les poids du modèle sont sauvegardés (best_model_weights) et le compteur de patience est remis à zéro ; sinon, le compteur augmente. Lorsque la patience (ici 3) est atteinte, l'entraînement s'arrête.
- À la fin, les **meilleurs poids rencontrés** sont restaurés sur le modèle, et non ceux de la toute dernière epoch, qui pourraient correspondre à un léger sur-apprentissage.

À RETENIR - Pourquoi surveiller la perte de validation plutôt que celle d'entraînement ?

La perte d'entraînement diminue presque toujours au fil des epochs, même quand le modèle commence à sur-apprendre les exemples d'entraînement par cœur. La perte de validation, calculée sur des données que le modèle n'a jamais utilisées pour apprendre, est un indicateur beaucoup plus fiable du moment où il faut arrêter l'entraînement.

Cellule 19 - Instanciation du modèle, de la perte et de l'optimiseur

CE QUE FAIT CETTE CELLULE

Cette cellule crée une instance du modèle d'attention croisée et configure les outils d'entraînement : fonction de perte, optimiseur et calendrier de taux d'apprentissage.

LE CODE

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

cross_att_model = CrossAttentionClassifier().to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.AdamW(cross_att_model.parameters(), lr=1e-5, weight_decay=1e-2)
# Scheduler forces learning rate to smoothly decay
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)

# Clear cache just to be safe
if torch.cuda.is_available():
    torch.cuda.empty_cache()
```

EXPLICATION DÉTAILLÉE

- **BCEWithLogitsLoss** est la fonction de perte adaptée à une classification binaire : elle combine une fonction sigmoïde et l'entropie croisée binaire, de façon numériquement stable.
- **AdamW** est une variante de l'algorithme d'optimisation Adam qui gère séparément la régularisation (weight_decay), ce qui limite le risque de sur-apprentissage en pénalisant les poids trop grands.

- **lr=1e-5** fixe un taux d'apprentissage faible, adapté à un modèle dont une grande partie (l'encodeur BiomedCLIP, ici gelé) n'est pas réentraînée.
- **CosineAnnealingLR** fait décroître progressivement le taux d'apprentissage selon une courbe en cosinus au fil des epochs, ce qui aide le modèle à affiner ses poids en douceur vers la fin de l'entraînement.

Cellule 20 - Résumé de l'architecture du modèle

CE QUE FAIT CETTE CELLULE

Cette cellule affiche un résumé de l'architecture du modèle d'attention croisée et de son nombre de paramètres.

LE CODE

```
summary(cross_att_model)
```

EXPLICATION DÉTAILLÉE

- **summary(...)**, fourni par torchinfo, affiche la liste des couches du modèle, leurs dimensions et le nombre de paramètres entraînaables de chacune - un moyen rapide de vérifier que l'architecture correspond bien à ce qui était prévu avant de lancer un entraînement potentiellement long.

Cellule 21 - Lancement de l'entraînement de l'attention croisée

CE QUE FAIT CETTE CELLULE

Cette cellule lance effectivement l'entraînement du modèle d'attention croisée, pour un maximum de 50 epochs.

LE CODE

```
# Launch the training block
history = cross_attention_train(
    model=cross_att_model,
    train_loader=cross_att_train_loader,
    val_loader=cross_att_val_loader,
    criterion=criterion,
    optimizer=optimizer,
    scheduler = scheduler,
    epochs=50
)
```

EXPLICATION DÉTAILLÉE

- Cet appel exécute la fonction définie à la cellule 18, avec le modèle, les chargeurs de données et les outils d'optimisation préparés aux cellules précédentes.
- L'historique (history) retourné contient, pour chaque epoch effectivement réalisée, la perte et l'exactitude sur l'entraînement et sur la validation - il sera utilisé pour tracer la courbe d'apprentissage à la cellule suivante.

À RETENIR - Résultat obtenu

L'entraînement s'arrête par arrêt anticipé à l'epoch 18 (sur 50 prévues), les meilleurs poids étant ceux de l'epoch 15 (perte de validation environ 0,40). À cette epoch, l'exactitude d'entraînement atteint environ 0,85 et l'exactitude de validation environ 0,82. Le notebook ne calcule pas, pour cette approche, de rapport de classification final (précision, rappel, F1 sur le test) : seules l'exactitude et la perte de validation par epoch sont disponibles, ce qui est suffisant pour situer la performance de cette approche entre la régression logistique et l'approche suivante par LoRA.

Cellule 22 - Visualisation de la courbe d'apprentissage**CE QUE FAIT CETTE CELLULE**

Cette cellule trace l'évolution de la perte d'entraînement et de validation au fil des epochs, pour visualiser la dynamique de l'apprentissage.

LE CODE

```
# Plot the training and validation loss
plt.plot(history["loss"], label="Training Loss")
plt.plot(history["val_loss"], label="Validation Loss")
# Add title and axis labels
plt.title("Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
# Show the legend
plt.legend()
# Display the plot
plt.show()
```

EXPLICATION DÉTAILLÉE

- Les deux courbes (perte d'entraînement et perte de validation) sont tracées sur le même graphique, ce qui permet de repérer visuellement un éventuel sur-apprentissage : si la perte d'entraînement continue de baisser alors que la perte de validation remonte, c'est le signe que le modèle commence à apprendre les exemples d'entraînement par cœur plutôt qu'à généraliser.

Cellule 23 - Quatrième approche : préparation des données pour LoRA**CE QUE FAIT CETTE CELLULE**

Cette cellule recopie le découpage entraînement / test déjà utilisé pour l'attention croisée, en vue de la quatrième et dernière approche : l'ajustement léger de BiomedCLIP par LoRA.

LE CODE

```
lora_X_train, lora_X_test, lora_y_train, lora_y_test = X_train_cross_att.copy(), \
    X_test_cross_att.copy(), y_train_cross_att.copy(), y_test_cross_att.copy()
lora_X_train = lora_X_train.reset_index(drop=True)
lora_X_test = lora_X_test.reset_index(drop=True)
lora_y_train = lora_y_train.reset_index(drop=True)
```

```
lora_y_test = lora_y_test.reset_index(drop=True)
```

EXPLICATION DÉTAILLÉE

- L'approche LoRA réutilise exactement le même échantillon réduit (22 000 paires) et le même découpage que l'attention croisée, ce qui permet de comparer les deux approches sur une base strictement identique.
- **.copy()** crée des copies indépendantes des dataframes, pour éviter que des modifications ultérieures sur les variables `lora_X_train` n'affectent par effet de bord les variables `X_train_cross_att` d'origine.
- **reset_index(drop=True)** renumérote les lignes de façon continue à partir de 0, ce qui sera utile pour l'accès par position dans la classe Dataset définie plus loin.

Cellule 24 - Chargement d'une seconde instance de BiomedCLIP, modifiable

CE QUE FAIT CETTE CELLULE

Cette cellule recharge BiomedCLIP, mais cette fois avec ses transformations d'image d'entraînement et de validation séparées, en vue de rendre une partie de ses poids entraînaibles.

LE CODE

```
clip_lora_model, lora_preprocess_train, lora_preprocess_val = create_model_and_transforms(
    'hf-hub:microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224')
lora_tokenizer = get_tokenizer(
    'hf-hub:microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224')
```

EXPLICATION DÉTAILLÉE

- **create_model_and_transforms(...)**, à la différence de `create_model_from_pretrained` utilisé à la cellule 4, retourne en plus deux fonctions de prétraitement distinctes : une pour l'entraînement (qui peut inclure des transformations aléatoires, comme de légères rotations, utiles pour la régularisation) et une pour la validation (déterministe).
- Cette nouvelle instance du modèle (`clip_lora_model`) est chargée séparément de celle de la cellule 4, car elle va être modifiée par l'ajout d'adaptateurs LoRA, alors que le modèle original doit rester intact pour les approches précédentes.

Cellule 25 - Classe Dataset pour le chargement à la volée

CE QUE FAIT CETTE CELLULE

Cette cellule définit une classe Dataset PyTorch qui charge et prétraite chaque image et chaque texte au moment où ils sont demandés, plutôt que de tout précalculer à l'avance.

LE CODE

```
class LoraDataset(Dataset):
    def __init__(self, dataframe, labels):
        self.dataframe = dataframe
        self.labels = labels

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
```

```

# Load raw data
raw_image_path = "./CheXpert/CheXpert-v1.0-small/" + self.dataframe.loc[idx,
"path_to_image"]
raw_report_text = self.dataframe.loc[idx, "report"]
label = self.labels[idx]

# Transform raw pixels into a 3x224x224 tensor
pil_img = Image.open(raw_image_path).convert("RGB")
processed_image_tensor = lora_preprocess_train(pil_img)

# Transform raw string into a 256-long token ID tensor
tokenized_text_tensor = lora_tokenizer(raw_report_text, context_length=256).squeeze(0)

return processed_image_tensor, tokenized_text_tensor, torch.tensor(label,
dtype=torch.float32)

```

EXPLICATION DÉTAILLÉE

- Une classe Dataset PyTorch doit définir deux méthodes : `__len__`, qui indique le nombre total d'exemples, et `__getitem__`, qui retourne un exemple à partir de sa position.
- Contrairement aux approches précédentes, où toutes les représentations étaient précalculées en une fois (cellules 14 et 15), ici l'image brute et le texte brut ne sont chargés et transformés qu'au moment de l'entraînement, exemple par exemple.
- Ce choix s'explique par le fait que l'on va maintenant réentraîner une partie de l'encodeur d'image lui-même via LoRA : il faut donc repartir des pixels bruts à chaque epoch, et non de représentations figées calculées une fois pour toutes avec l'ancien modèle.

À RETENIR - Chargement à la volée plutôt que précalcul

Précalculer toutes les représentations à l'avance (comme aux cellules 14 et 15) est plus rapide à l'entraînement, mais suppose que l'encodeur reste figé. Dès qu'une partie de l'encodeur doit être réentraînée, comme c'est le cas avec LoRA, il devient nécessaire de repartir des données brutes à chaque passage, via une classe Dataset classique.

Cellule 26 - Construction des chargeurs de données pour LoRA

CE QUE FAIT CETTE CELLULE

Cette cellule instancie la classe Dataset définie précédemment pour l'entraînement et le test, puis prépare les chargeurs de lots correspondants.

LE CODE

```

lora_train_dataset = LoraDataset(lora_X_train, lora_y_train)
lora_test_dataset = LoraDataset(lora_X_test, lora_y_test)

lora_train_loader = DataLoader(lora_train_dataset, batch_size=32, shuffle=True)
lora_val_loader = DataLoader(lora_test_dataset, batch_size=32, shuffle=False)

```

EXPLICATION DÉTAILLÉE

- La logique est la même que pour l'attention croisée (cellule 16), avec une taille de lot plus petite (**batch_size=32** au lieu de 64) : réentraîner une partie de l'encodeur consomme davantage de mémoire

graphique que la seule attention croisée sur des représentations déjà calculées, ce qui impose des lots plus modestes.

Cellule 27 - Tête de classification pour LoRA

CE QUE FAIT CETTE CELLULE

Cette cellule définit la tête de classification utilisée au-dessus de l'encodeur ajusté par LoRA : il s'agit exactement de la même architecture d'attention croisée que celle définie à la cellule 17.

LE CODE

```
class LoraClassifier(nn.Module):
    def __init__(self, embed_dim=768, num_heads=8, dropout=0.3):
        super().__init__()
        self.cross_attention = nn.MultiheadAttention(
            embed_dim=embed_dim, num_heads=num_heads,
            dropout=dropout, batch_first=True
        )
        self.layer_norm1 = nn.LayerNorm(embed_dim)
        self.layer_norm2 = nn.LayerNorm(embed_dim)
        self.ffn = nn.Sequential(
            nn.Linear(embed_dim, embed_dim * 2), nn.GELU(),
            nn.Dropout(dropout), nn.Linear(embed_dim * 2, embed_dim)
        )
        self.classifier = nn.Sequential(
            nn.Linear(embed_dim, 256), nn.GELU(), nn.LayerNorm(256),
            nn.Dropout(dropout), nn.Linear(256, 1)
        )

    def forward(self, img_patches, text_tokens):
        norm_text = self.layer_norm1(text_tokens)
        attn_output, _ = self.cross_attention(
            query=norm_text, key=img_patches, value=img_patches)
        x = (attn_output + text_tokens)
        x = self.layer_norm2(self.ffn(x)) + x
        x_pooled, _ = torch.max(x, dim=1)
        return self.classifier(x_pooled)
```

EXPLICATION DÉTAILLÉE

- Cette classe est identique, ligne pour ligne, à CrossAttentionClassifier de la cellule 17 (seul le nom change). Elle est redéfinie séparément ici pour que cette quatrième approche reste autonome et puisse être exécutée indépendamment des cellules précédentes.
- Ce qui change réellement entre l'approche 3 (cellule 17) et l'approche 4, ce n'est donc pas cette tête de classification, mais la façon dont les représentations d'image et de texte qui l'alimentent sont produites : à la cellule 17, elles proviennent d'un encodeur gelé ; ici, elles proviendront d'un encodeur partiellement réentraîné par LoRA (cellule suivante).

Cellule 28 - Encapsulation de BiomedCLIP avec des adaptateurs LoRA

CE QUE FAIT CETTE CELLULE

Cette cellule définit le modèle complet de la quatrième approche : elle enveloppe BiomedCLIP dans des adaptateurs LoRA entraînaables, puis branche la tête d'attention croisée sur ses représentations internes.

LE CODE

```
class BioMedCLIPWithLoRA(nn.Module):
    def __init__(self, biomedclip_base_model, r=8, alpha=16):
        super().__init__()

        # Target the Attention projections of BioMedCLIP.
        lora_config = LoraConfig(
            r=r,
            lora_alpha=alpha,
            target_modules=["qkv", "query", "value"],
            lora_dropout=0.1,
            bias="none"
        )

        # Wrap the base model. PEFT automatically freezes base weights
        # and injects the trainable A and B matrices into the target_modules.
        self.biomedclip = get_peft_model(biomedclip_base_model, lora_config)

        # Instantiate custom fusion head
        self.fusion_head = LoraClassifier()

    def forward(self, image_tensors, token_id_tensors):
        text_outputs = self.biomedclip.base_model.model.text.transformer(token_id_tensors)
        text_tokens = text_outputs.last_hidden_state

        img_patches =
        self.biomedclip.base_model.model.visual.trunk.forward_features(image_tensors)

        if len(img_patches.shape) == 2:
            x = self.biomedclip.base_model.model.visual.trunk.patch_embed(image_tensors)
            x = self.biomedclip.base_model.model.visual.trunk._pos_embed(x)
            x = self.biomedclip.base_model.model.visual.trunk.patch_drop(x)
            x = self.biomedclip.base_model.model.visual.trunk.norm_pre(x)
            img_patches = self.biomedclip.base_model.model.visual.trunk.blocks(x)
            img_patches = self.biomedclip.base_model.model.visual.trunk.norm(img_patches)

        return self.fusion_head(img_patches, text_tokens)
```

EXPLICATION DÉTAILLÉE

- **LoraConfig(...)** définit comment les adaptateurs LoRA seront injectés : $r=8$ fixe le rang des matrices d'adaptation (plus il est petit, moins il y a de nouveaux paramètres à entraîner), et `target_modules` désigne précisément les couches de projection d'attention (qkv, query, value) qui recevront ces adaptateurs.
- **get_peft_model(...)** enveloppe le modèle BiomedCLIP d'origine : tous ses poids d'origine sont gelés (non modifiables), et seules de petites matrices supplémentaires (les adaptateurs LoRA) sont ajoutées et rendues entraînaables sur les couches ciblées.

- La méthode `forward` extrait les représentations détaillées de texte et d'image de la même façon que `biomed_clip_cross_att` (cellule 13), mais en passant cette fois par le modèle enveloppé (`self.biomedclip`), dont une petite partie des poids évoluera pendant l'entraînement.
- Le bloc `if len(img_patches.shape) == 2` est une sécurité technique : selon la configuration interne du modèle, l'appel direct peut parfois retourner un vecteur déjà résumé (2 dimensions) plutôt que la grille complète de patches (3 dimensions) ; dans ce cas, le code recalcule manuellement la grille de patches en remontant les couches internes une par une.
- Enfin, `self.fusion_head(...)` applique la tête de classification de la cellule 27 sur ces représentations, désormais partiellement réentraînables.

À RETENIR - Qu'est-ce que LoRA ?

LoRA (Low-Rank Adaptation) est une technique d'ajustement léger d'un grand modèle pré-entraîné : plutôt que de réentraîner l'intégralité de ses millions de paramètres, on gèle le modèle d'origine et on lui ajoute de petites matrices entraînaibles, de rang réduit, sur certaines couches ciblées. Cela permet d'adapter le modèle à une nouvelle tâche en entraînant beaucoup moins de paramètres, plus vite et avec moins de mémoire, tout en conservant l'essentiel des connaissances acquises lors du pré-entraînement.

Cellule 29 - Fonction d'entraînement avec précision mixte

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction d'entraînement pour le modèle LoRA. Elle reprend la même logique que `cross_attention_train` (cellule 18), enrichie d'un calcul en précision mixte pour gérer la charge supplémentaire liée au réentraînement partiel de l'encodeur.

LE CODE

```
def lora_train(model, train_loader, val_loader, criterion, optimizer, scheduler,
              epochs=100, patience=3):
    history = {'loss': [], 'val_loss': [], 'accuracy': [], 'val_accuracy': []}
    scaler = GradScaler('cuda')
    best_val_loss = float('inf')
    patience_counter = 0
    best_model_weights = None
    best_epoch = 0

    for epoch in range(epochs):
        model.train()
        total_loss, correct = 0, 0

        for inputs_img, inputs_txt, labels in train_loader:
            inputs_img = inputs_img.to(device, non_blocking=True)
            inputs_txt = inputs_txt.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)
            optimizer.zero_grad(set_to_none=True)
            with autocast('cuda'):
                outputs = model(inputs_img, inputs_txt).squeeze(1)
                loss = criterion(outputs, labels)
            scaler.scale(loss).backward()
```

```
        scaler.step(optimizer)
        scaler.update()

        total_loss += loss.item()
        correct += ((torch.sigmoid(outputs) >= 0.5).float() == labels).sum().item()

    train_loss = total_loss / len(train_loader)
    train_acc = correct / len(train_loader.dataset)

    model.eval()
    val_loss, val_correct = 0, 0
    with torch.no_grad():
        for inputs_img, inputs_txt, labels in val_loader:
            inputs_img = inputs_img.to(device, non_blocking=True)
            inputs_txt = inputs_txt.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)
            with autocast('cuda'):
                outputs = model(inputs_img, inputs_txt).squeeze(1)
                loss = criterion(outputs, labels)
            val_loss += loss.item()
            val_correct += ((torch.sigmoid(outputs) >= 0.5).float() == labels).sum().item()

    val_loss /= len(val_loader)
    val_acc = val_correct / len(val_loader.dataset)
    scheduler.step()
    current_lr = optimizer.param_groups[0]['lr']

    history['loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['accuracy'].append(train_acc)
    history['val_accuracy'].append(val_acc)

    print(f"Epoch [{epoch+1}/{epochs}], Loss: {train_loss:.4f}, Acc: {train_acc:.4f}, "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}, LR: {current_lr:.6f}")

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        best_epoch = epoch + 1
        best_model_weights = copy.deepcopy(model.state_dict())
    else:
        patience_counter += 1

    if patience_counter >= patience:
        print(f"Early stopping triggered! Stopping training at epoch {epoch+1}.")
        break

    if best_model_weights is not None:
        print(f"Restoring best model weights found at Epoch {best_epoch}")
        model.load_state_dict(best_model_weights)

    return history
```

EXPLICATION DÉTAILLÉE

- La structure générale (boucle d'entraînement, boucle de validation, arrêt anticipé) est rigoureusement la même qu'à la cellule 18. La différence porte sur la gestion de la précision numérique.
- **GradScaler** et **autocast('cuda')** mettent en œuvre la précision mixte automatique : certaines opérations sont calculées en précision réduite (16 bits au lieu de 32), ce qui accélère les calculs et réduit la mémoire utilisée, tandis que GradScaler compense le risque de perte de précision sur le calcul du gradient en mettant à l'échelle la perte avant la rétropropagation.
- **non_blocking=True** lors du transfert des tenseurs vers la carte graphique permet de chevaucher ce transfert avec d'autres calculs, pour gagner un peu de temps.

À RETENIR - Pourquoi la précision mixte est-elle utile ici en particulier ?

Les trois approches précédentes n'entraînaient qu'une petite tête de classification au-dessus d'un encodeur gelé. Ici, des adaptateurs LoRA sont entraînés à l'intérieur même de l'encodeur, ce qui implique de conserver en mémoire davantage d'informations pour le calcul du gradient. La précision mixte est une technique standard pour rendre ce type d'entraînement plus léger, sans changer la logique d'apprentissage elle-même.

Cellule 30 - Instanciation du modèle LoRA

CE QUE FAIT CETTE CELLULE

Cette cellule crée une instance du modèle BiomedCLIP encapsulé avec LoRA, et configure ses outils d'entraînement.

LE CODE

```
gc.collect()
torch.cuda.empty_cache()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

lora_model = BioMedCLIPWithLoRA(biomedclip_base_model=clip_lora_model, r=8,
alpha=16).to(device)
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.AdamW(lora_model.parameters(), lr=1e-5, weight_decay=1e-2)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)
```

EXPLICATION DÉTAILLÉE

- **gc.collect()** et **torch.cuda.empty_cache()** libèrent la mémoire encore occupée par les modèles et tenseurs des approches précédentes, avant de charger ce nouveau modèle, plus gourmand en mémoire.
- Les réglages de la fonction de perte, de l'optimiseur et du calendrier de taux d'apprentissage sont identiques à ceux de l'attention croisée (cellule 19), pour permettre une comparaison équitable entre les deux approches.

Cellule 31 - Résumé de l'architecture du modèle LoRA

CE QUE FAIT CETTE CELLULE

Cette cellule affiche un résumé de l'architecture du modèle encapsulé avec LoRA.

LE CODE

```
summary(lora_model)
```

EXPLICATION DÉTAILLÉE

- Ce résumé permet notamment de distinguer, dans le décompte des paramètres, ceux qui sont gelés (les poids d'origine de BiomedCLIP, très nombreux) de ceux qui sont entraînaables (les adaptateurs LoRA et la tête de classification, beaucoup moins nombreux) - une vérification utile pour confirmer que l'ajustement reste bien léger.

Cellule 32 - Lancement de l'entraînement LoRA

CE QUE FAIT CETTE CELLULE

Cette cellule lance l'entraînement du modèle LoRA, pour un maximum de 50 epochs.

LE CODE

```
# Launch the training block
history_lora = lora_train(
    model=lora_model,
    train_loader=lora_train_loader,
    val_loader=lora_val_loader,
    criterion=criterion,
    optimizer=optimizer,
    scheduler = scheduler,
    epochs=50
)
```

EXPLICATION DÉTAILLÉE

- Cet appel suit exactement le même schéma que celui de la cellule 21 pour l'attention croisée, avec la fonction `lora_train` et le modèle LoRA préparés aux cellules précédentes.

À RETENIR - Résultat obtenu

L'entraînement s'arrête par arrêt anticipé à l'epoch 17 (sur 50 prévues), les meilleurs poids étant ceux de l'epoch 14 (perte de validation environ 0,35). À cette epoch, l'exactitude d'entraînement atteint environ 0,86 et l'exactitude de validation environ 0,84, supérieure à celle obtenue avec l'attention croisée seule (environ 0,82). Comme pour l'approche précédente, le notebook ne calcule pas de rapport de classification final avec précision, rappel et F1 sur le test : l'amélioration observée ici porte sur l'exactitude de validation au fil de l'entraînement, et confirme que réentraîner une petite partie de l'encodeur, même légèrement, apporte un gain mesurable par rapport à un encodeur totalement gelé.

Cellule 33 - Visualisation de la courbe d'apprentissage de LoRA

CE QUE FAIT CETTE CELLULE

Cette dernière cellule du notebook trace l'évolution de la perte d'entraînement et de validation du modèle LoRA, au fil des epochs.

LE CODE

```
# Plot the training and validation loss
plt.plot(history_lora["loss"], label="Training Loss")
plt.plot(history_lora["val_loss"], label="Validation Loss")
# Add title and axis labels
plt.title("Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
# Show the legend
plt.legend()
# Display the plot
plt.show()
```

EXPLICATION DÉTAILLÉE

- Cette cellule est identique dans sa logique à la cellule 22, appliquée cette fois à l'historique d'entraînement du modèle LoRA (history_lora), pour visualiser sa propre dynamique d'apprentissage.

Synthèse - les quatre approches BiomedCLIP

Le tableau ci-dessous récapitule les résultats obtenus sur le jeu de test pour chacune des quatre approches. Pour l'attention croisée et l'ajustement par LoRA, seules l'exactitude de validation et la perte à la meilleure epoch sont disponibles, le notebook ne calculant pas de rapport de classification final pour ces deux approches plus coûteuses en temps de calcul.

Approche	Échantillon	ROC-AUC (test)	Exactitude (test)	F1 (test)
Similarité cosinus	36 000 paires	0,66	0,62	0,59
Régression logistique	36 000 paires	0,83	0,75	0,75 (moyen)
Attention croisée + MLP	22 000 paires	n.d.*	0,82*	n.d.*
Ajustement LoRA	22 000 paires	n.d.*	0,84*	n.d.*

* Exactitude de validation à la meilleure epoch (arrêt anticipé), et non un rapport de classification complet calculé une fois pour toutes sur le jeu de test.

PARTIE C

Modélisation avec BioViL-T - le modèle retenu

Notebook : *models_biovil_t*. BioViL-T est un modèle pré-entraîné spécifiquement pour la radiographie thoracique et ses comptes rendus, plutôt que pour l'imagerie médicale en général : il associe un encodeur d'image dédié à la radiographie thoracique et CXR-BERT, un modèle de langage entraîné sur des comptes rendus radiologiques. Ce notebook reprend la même démarche en deux temps que la partie B (similarité, puis attention croisée), et c'est la seconde approche, l'attention croisée, qui constitue le modèle finalement retenu pour le projet.

Cellule 1 - Importation des bibliothèques

CE QUE FAIT CETTE CELLULE

Cette cellule charge l'ensemble des outils nécessaires à ce notebook, en ajoutant par rapport à la partie B les bibliothèques spécifiques à BioViL-T.

LE CODE

```
import pandas as pd
import numpy as np
import gc
import os
from tqdm import tqdm
from pathlib import Path
import copy

from sklearn.model_selection import train_test_split
import torch
from PIL import Image
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score, accuracy_score, precision_score, \
    recall_score, f1_score, confusion_matrix, classification_report
from transformers import AutoModel, AutoTokenizer, AutoProcessor, AutoModelForMaskedLM
from torch.utils.data import TensorDataset, DataLoader, Dataset
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import plotly as px
from plotly import graph_objects as go
from torchinfo import summary

from health_multimodal.image.inference_engine import ImageInferenceEngine
from health_multimodal.image.model.pretrained import get_biovil_t_image_encoder
from health_multimodal.image.data.transforms import create_chest_xray_transform_for_inference
```

EXPLICATION DÉTAILLÉE

- **tqdm** et **os** seront utilisés ici pour de vraies barres de progression sur des boucles longues, et pour créer des dossiers sur le disque - ce notebook va en effet sauvegarder des fichiers intermédiaires, contrairement à la partie B.
- **health_multimodal** est la bibliothèque officielle de Microsoft pour BioViL-T : *ImageInferenceEngine*, *get_biovil_t_image_encoder* et *create_chest_xray_transform_for_inference* fournissent respectivement un

moteur d'inférence prêt à l'emploi, l'encodeur d'image pré-entraîné, et la fonction de prétraitement d'image spécifiquement calibrée pour des radiographies thoraciques.

- **AutoModel** et **AutoTokenizer**, de la bibliothèque transformers, serviront cette fois à charger CXR-BERT, l'encodeur de texte de BioViL-T, de façon similaire à la manière dont PubMedBERT était chargé indirectement via BiomedCLIP dans la partie B.

Cellule 2 - Chargement du jeu de données préparé

CE QUE FAIT CETTE CELLULE

Cette cellule charge exactement le même fichier que la partie B : les 36 000 paires construites dans la partie A.

LE CODE

```
pd.options.display.max_columns = None
dataset_sample = pd.read_csv("../chexpert_plus_dataset_sample.csv", index_col=0)
dataset_sample
```

EXPLICATION DÉTAILLÉE

- Réutiliser le même fichier que pour BiomedCLIP garantit que les deux familles de modèles sont comparées sur un pied d'égalité, avec exactement les mêmes paires.

Cellule 3 - Découpage en jeu d'entraînement et de test

CE QUE FAIT CETTE CELLULE

Cette cellule effectue le même découpage stratifié 80/20 que dans la partie B, sur l'intégralité des 36 000 paires (sans réduction d'échantillon, contrairement à l'attention croisée et à LoRA pour BiomedCLIP).

LE CODE

```
X = dataset_sample.drop(columns=["target"])
y = dataset_sample["target"]

# Split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

EXPLICATION DÉTAILLÉE

- Le découpage produit 28 800 paires d'entraînement et 7 200 paires de test, dans les mêmes proportions que pour les approches BiomedCLIP du début de la partie B.
- Contrairement à l'attention croisée BiomedCLIP (partie B, cellule 12), BioViL-T n'a pas eu besoin de réduire la taille de l'échantillon : son encodeur d'image, plus léger et plus spécialisé, permet de travailler directement sur l'ensemble des 36 000 paires.

Cellule 4 - Chargement des deux encodeurs BioViL-T

CE QUE FAIT CETTE CELLULE

Cette cellule charge les deux composants de BioViL-T : CXR-BERT pour le texte, et l'encodeur d'image dédié à la radiographie thoracique.

LE CODE

```
# Setup device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Load the comprehensive BioViL-T repo for Text
model_id = "microsoft/BiomedVLP-BioViL-T"
# Specialized CXR-BERT tokenizer and model
tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
text_model = AutoModel.from_pretrained(model_id, trust_remote_code=True).to(device)

text_model.eval()

# Instantiate the BioViL-T Image Engine
image_model = get_biovil_t_image_encoder().to(device)
image_transform = create_chest_xray_transform_for_inference(resize=512, center_crop_size=448)

image_model.eval()
```

EXPLICATION DÉTAILLÉE

- **AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)** charge CXR-BERT, le modèle de langage de BioViL-T, entraîné spécifiquement sur des comptes rendus de radiographies thoraciques - à la différence de PubMedBERT (partie B), entraîné sur la littérature biomédicale en général.
- **trust_remote_code=True** autorise l'exécution de code personnalisé fourni avec le modèle sur le Hub Hugging Face, nécessaire ici car BioViL-T définit des méthodes spécifiques (comme `get_projected_text_embeddings`, utilisée plus loin).
- **get_biovil_t_image_encoder()** charge l'encodeur d'image pré-entraîné spécifiquement sur des radiographies thoraciques.
- **create_chest_xray_transform_for_inference(resize=512, center_crop_size=448)** prépare la fonction de prétraitement d'image adaptée à cet encodeur : redimensionnement à 512 pixels puis recadrage central à 448 pixels.
- Les deux modèles sont placés en **mode évaluation** (`eval()`), comme dans la partie B.

À RETENIR - Un modèle spécialisé plutôt que généraliste

BiomedCLIP (partie B) a été entraîné sur de grandes quantités d'images et de textes biomédicaux variés. BioViL-T, à l'inverse, a été entraîné spécifiquement sur des radiographies thoraciques et leurs comptes rendus, ce qui lui donne un avantage attendu sur cette tâche précise, au prix d'une moindre généralité.

Cellule 5 - Affichage de l'architecture du modèle de texte

CE QUE FAIT CETTE CELLULE

Cette cellule affiche l'architecture complète de CXR-BERT, pour inspection.

LE CODE

```
text_model
```

EXPLICATION DÉTAILLÉE

- Comme pour toute cellule Jupyter se terminant par le nom d'une variable, Python affiche automatiquement sa représentation textuelle : ici, la liste de toutes les couches du modèle de langage CXR-BERT.

Cellule 6 - Fonctions d'encodage du texte et de l'image

CE QUE FAIT CETTE CELLULE

Cette cellule définit les deux fonctions d'encodage centrales du notebook, qui extraient à la fois un vecteur global (pour la similarité) et des représentations détaillées (pour l'attention croisée), en un seul passage dans chaque modèle.

LE CODE

```
# Test of XCR-BERT
def get_text_embeddings(report_text):
    inputs = tokenizer(
        report_text,
        padding="max_length",
        truncation=True,
        max_length=512,
        return_tensors="pt"
    ).to(device)

    with torch.no_grad():
        # Get the joint-space projection (128-d vector) for phase 1 similarity matching
        text_embeddings = text_model.get_projected_text_embeddings(
            input_ids=inputs.input_ids,
            attention_mask=inputs.attention_mask
        )

        # To get the raw token hidden states for Phase 2 (Cross-Attention)
        outputs = text_model(
            input_ids=inputs.input_ids,
            attention_mask=inputs.attention_mask,
            return_dict=True
        )
        sequence_outputs = outputs.last_hidden_state

    # Normalize joint-space vector for cosine similarity
    text_embeddings = F.normalize(text_embeddings, p=2, dim=-1)
    return text_embeddings, sequence_outputs # Shape: [1, 128], [1, 512, 768]

def get_image_embeddings(image_path):
```

```
#Extracts true BioViL-T spatial and global image embeddings.
raw_image = Image.open(image_path).convert("L")
processed_tensor = image_transform(raw_image).unsqueeze(0).to(device)

with torch.no_grad():
    image_outputs = image_model(processed_tensor)

    # Pull global 128-d vector and spatial patch tokens
    global_img_emb = image_outputs.projected_global_embedding
    patch_img_emb = image_outputs.projected_patch_embeddings

    # L2 normalize global vector for cosine similarity
    global_img_emb = F.normalize(global_img_emb, p=2, dim=-1)
    return global_img_emb, patch_img_emb # Shape: [1, 128], [1, 128, 14, 14]
```

EXPLICATION DÉTAILLÉE

- **get_text_embeddings** tokenise le rapport avec un remplissage systématique à 512 tokens (padding="max_length"), puis récupère deux choses en un seul appel au modèle : **get_projected_text_embeddings** retourne un vecteur global de 128 dimensions, conçu pour la similarité (phase 1), tandis que l'appel direct au modèle retourne **last_hidden_state**, la séquence complète des représentations de chaque token, destinée à l'attention croisée (phase 2).
- **get_image_embeddings** ouvre l'image en niveaux de gris (**convert("L")**), puisqu'une radiographie thoracique est une image monochrome, contrairement aux images RGB génériques utilisées par BiomedCLIP. Elle retourne de la même façon un vecteur global (**projected_global_embedding**, 128 dimensions) et une grille de patches spatiaux (**projected_patch_embeddings**, 128 canaux sur une grille 14x14).
- Les deux vecteurs globaux sont normalisés (**F.normalize**) pour permettre un calcul de similarité cosinus, exactement comme dans la partie B.

À RETENIR - Deux usages, un seul passage dans le modèle

Plutôt que d'appeler le modèle deux fois (une fois pour la similarité, une fois pour l'attention croisée), ces fonctions récupèrent en un seul passage à la fois le résumé global et les représentations détaillées. C'est une optimisation utile, puisque l'extraction sera réutilisée pour les deux phases du notebook, comme on le voit à la cellule suivante.

Cellule 7 - Extraction et sauvegarde des représentations sur le jeu d'entraînement

CE QUE FAIT CETTE CELLULE

Cette cellule calcule, pour chaque paire du jeu d'entraînement, le score de similarité ET sauvegarde sur disque les représentations détaillées, qui seront réutilisées plus loin pour l'attention croisée, sans avoir à refaire ce calcul coûteux une seconde fois.

LE CODE

```
class BiovilDataset(Dataset):
    def __init__(self, df, base_path="./CheXpert/CheXpert-v1.0-small/"):
        self.df = df.reset_index(drop=True)
```

```

        self.base_path = base_path

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.base_path + self.df.loc[idx, "path_to_image"]
        report = self.df.loc[idx, "report"]
        return img_path, report, idx

# Setup folders to store heavy tensors for Phase 2 Cross-Attention
os.makedirs("./features/image_patches_train", exist_ok=True)
os.makedirs("./features/text_sequences_train", exist_ok=True)

train_dataset = BiovilDataset(X_train)
train_loader = DataLoader(train_dataset, batch_size=1, shuffle=False)

similarity_scores = np.zeros(len(X_train))

print("Extracting features and calculating similarities...")
for img_paths, reports, indices in tqdm(train_loader):
    img_path = img_paths[0]
    report = reports[0]
    idx = indices[0].item()

    img_vector, img_patches = get_image_embeddings(img_path)
    text_vector, text_sequence = get_text_embeddings(report)

    sim = torch.mm(img_vector, text_vector.t()).cpu().item()
    similarity_scores[idx] = sim

    # Save heavy spatial/token features to disk using the unique index
    torch.save(img_patches.detach().cpu(), f"./features/image_patches_train/{idx}.pt")
    torch.save(text_sequence.cpu().detach(), f"./features/text_sequences_train/{idx}.pt")

# Write results back to DataFrame
X_train["similarity_score"] = similarity_scores

```

EXPLICATION DÉTAILLÉE

- **BiovilDataset** est une classe Dataset minimale, qui se contente de retourner le chemin de l'image, le texte du rapport et son indice ; le chargement et le calcul lourds se font dans la boucle qui suit, pas dans la classe elle-même.
- **os.makedirs(..., exist_ok=True)** crée les dossiers qui accueilleront les fichiers de représentations sauvegardées, sans erreur s'ils existent déjà.
- **batch_size=1** traite les exemples un par un : les fonctions d'encodage définies à la cellule précédente attendent une seule image et un seul texte à la fois.
- Pour chaque paire, la boucle calcule la **similarité cosinus** entre les deux vecteurs globaux (comme dans la partie B), mais sauvegarde aussi sur disque, via **torch.save(...)**, les représentations détaillées (patches d'image et séquence de tokens de texte), nommées d'après l'indice de la paire.
- Cette double extraction en une seule passe évite de recalculer les représentations détaillées une seconde fois lors de la phase d'attention croisée, plus loin dans le notebook : un gain de temps important, puisque le passage dans les deux encodeurs est l'opération la plus coûteuse du notebook.

À RETENIR - Pourquoi sauvegarder sur disque plutôt qu'en mémoire ?

Garder en mémoire vive les représentations détaillées des 28 800 paires d'entraînement représenterait un volume considérable. Les écrire sur disque, fichier par fichier, permet de ne charger en mémoire que ce qui est nécessaire à un instant donné, au prix d'un accès disque un peu plus lent mais largement supportable.

Cellule 8 - Calcul du seuil optimal sur le jeu d'entraînement

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction d'évaluation du seuil optimal, identique dans sa logique à celle de la partie B, et l'applique aux similarités calculées sur le jeu d'entraînement.

LE CODE

```
# Function returning the similarity score threshold
def evaluate(targets, similarities):
    auc = roc_auc_score(targets, similarities)
    print(f"Test ROC-AUC Score: {auc:.4f}")

    fpr, tpr, thresholds = roc_curve(targets, similarities)

    j_scores = tpr - fpr
    best_idx = np.argmax(j_scores)
    best_threshold = thresholds[best_idx]

    print(f"Optimal Similarity Threshold: {best_threshold:.4f}")
    print(f"Maximized True Positive Rate (Sensitivity): {tpr[best_idx]:.4f}")
    print(f"Minimized False Positive Rate (1 - Specificity): {fpr[best_idx]:.4f}")

    plt.figure(figsize=(6, 5))
    plt.plot(fpr, tpr, label=f'BiomedCLIP Baseline (AUC = {auc:.2f})', color='darkorange', lw=2)
    plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
    plt.scatter(fpr[best_idx], tpr[best_idx], color='red',
                label=f'Best Threshold ({best_threshold:.3f})', zorder=5)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Receiver Operating Characteristic (ROC) Curve')
    plt.legend(loc="lower right")
    plt.grid(True, alpha=0.3)
    plt.show()

    return best_threshold

targets = y_train.to_numpy()
similarities = X_train["similarity_score"].to_numpy()
opt_threshold = evaluate(targets, similarities)
```

EXPLICATION DÉTAILLÉE

- Cette fonction est rigoureusement la même que celle de la partie B (cellule 6) : calcul du ROC-AUC, puis recherche du seuil optimal par le critère de Youden sur le jeu d'entraînement.
- Le libellé affiché ("BiomedCLIP Baseline") est conservé tel quel dans le code source du graphique, bien qu'il s'applique ici à BioViL-T : un détail mineur de copier-coller entre les deux notebooks, sans incidence sur le calcul lui-même.

À RETENIR - Résultat obtenu

Sur le jeu d'entraînement, la similarité BioViL-T atteint un ROC-AUC d'environ 0,86, avec un seuil optimal proche de 0,29 - une nette amélioration par rapport au ROC-AUC d'environ 0,66 obtenu avec la similarité BiomedCLIP dans la partie B, ce qui confirme l'intérêt d'un encodeur spécialisé pour la radiographie thoracique.

Cellule 9 - Fonction d'évaluation complète

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction qui calcule l'ensemble des métriques de classification à partir d'un seuil donné, identique dans sa logique à celle de la partie B.

LE CODE

```
# Display the metrics of the binary classification
def bin_classif_metrics(similarities, targets, opt_threshold, type='Train'):
    predictions = (similarities >= opt_threshold).astype(int)

    accuracy = accuracy_score(targets, predictions)
    precision = precision_score(targets, predictions)
    recall = recall_score(targets, predictions)
    f1 = f1_score(targets, predictions)

    tn, fp, fn, tp = confusion_matrix(targets, predictions).ravel()
    specificity = tn / (tn + fp)

    print(f"--- Operational Metrics at Threshold {opt_threshold} ---")
    print(f"{type} ROC-AUC Score: {roc_auc_score(targets, similarities):.4f}")
    print(f"{type} Accuracy: {accuracy:.4f}")
    print(f"{type} Precision: {precision:.4f}")
    print(f"{type} Recall: {recall:.4f}")
    print(f"{type} F1-Score: {f1:.4f}")
    print(f"{type} Specificity: {specificity:.4f} (FPR is {1-specificity:.4f})")
```

EXPLICATION DÉTAILLÉE

- Cette fonction est identique à celle de la partie B (cellule 7) : conversion du score continu en décision binaire au seuil donné, puis calcul des cinq métriques standard (exactitude, précision, rappel, F1, spécificité).

Cellule 10 - Extraction et sauvegarde des représentations sur le jeu de test

CE QUE FAIT CETTE CELLULE

Cette cellule reproduit, sur le jeu de test, exactement la même extraction et sauvegarde que la cellule 7 pour le jeu d'entraînement.

LE CODE

```
# Initialize dataset and loader for test set
test_dataset = BiovilDataset(X_test)
test_loader = DataLoader(test_dataset, batch_size=1, shuffle=False)

os.makedirs("./features/image_patches_test", exist_ok=True)
os.makedirs("./features/text_sequences_test", exist_ok=True)

similarity_scores = np.zeros(len(X_test))

print("Extracting features and calculating similarities...")
for img_paths, reports, indices in tqdm(test_loader):
    img_path = img_paths[0]
    report = reports[0]
    idx = indices[0].item()

    img_vector, img_patches = get_image_embeddings(img_path)
    text_vector, text_sequence = get_text_embeddings(report)

    sim = torch.mm(img_vector, text_vector.t()).cpu().item()
    similarity_scores[idx] = sim

    torch.save(img_patches.detach().cpu(), f"./features/image_patches_test/{idx}.pt")
    torch.save(text_sequence.cpu().detach(), f"./features/text_sequences_test/{idx}.pt")

# Write results back to DataFrame
X_test["similarity_score"] = similarity_scores
```

EXPLICATION DÉTAILLÉE

- Seul change le dossier de destination (image_patches_test au lieu de image_patches_train), pour garder les représentations d'entraînement et de test clairement séparées sur le disque.

Cellule 11 - Affichage des métriques de similarité, train et test

CE QUE FAIT CETTE CELLULE

Cette cellule applique la fonction d'évaluation aux deux jeux de données, avec le seuil calculé sur l'entraînement.

LE CODE

```
print("-----Train metrics-----")
targets = y_train.to_numpy()
similarities = X_train["similarity_score"].to_numpy()
bin_classif_metrics(similarities, targets, opt_threshold, type='Train')
```

```
print("-----Test metrics-----")
targets = y_test.to_numpy()
similarities = X_test["similarity_score"].to_numpy()
bin_classif_metrics(similarities, targets, opt_threshold, type='Test')
```

EXPLICATION DÉTAILLÉE

- Comme dans la partie B, le seuil utilisé pour le test est celui calculé exclusivement sur l'entraînement, ce qui garantit une évaluation honnête.

À RETENIR - Résultat obtenu - première approche BioViL-T

Sur le jeu de test, la similarité BioViL-T atteint un ROC-AUC de 0,86, une exactitude de 0,78, une précision de 0,77, un rappel de 0,79 et un F1 de 0,78. Ces résultats dépassent largement ceux de la similarité BiomedCLIP (ROC-AUC 0,66) et même ceux de la régression logistique BiomedCLIP (ROC-AUC 0,83), avec une approche pourtant plus simple - la spécialisation de l'encodeur pour la radiographie thoracique fait une réelle différence.

Cellule 12 - Fonctions de relecture des représentations sauvegardées

CE QUE FAIT CETTE CELLULE

Cette cellule définit deux fonctions qui relisent depuis le disque les représentations détaillées sauvegardées aux cellules 7 et 10, pour la phase d'attention croisée.

LE CODE

```
def load_saved_images(idx, type='train'):
    # Load image patches: expected shape [128, 14, 14]
    img_patches = torch.load(f"./features/image_patches_{type}/{idx}.pt",
                             weights_only=True).squeeze(0)
    return img_patches

def load_saved_texts(idx, type='train'):
    # Load text sequence: expected shape [512, 768]
    text_sequence = torch.load(f"./features/text_sequences_{type}/{idx}.pt",
                               weights_only=True).squeeze(0)
    return text_sequence[:256, :]
```

EXPLICATION DÉTAILLÉE

- **torch.load(..., weights_only=True)** recharge un tenseur précédemment sauvegardé ; `weights_only=True` restreint le chargement aux seules données numériques, une précaution de sécurité recommandée lorsqu'on charge des fichiers (même si, ici, ce sont des fichiers produits par le notebook lui-même).
- **load_saved_texts** tronque la séquence de tokens à ses 256 premiers (`[:256, :]`), alors qu'elle avait été sauvegardée sur 512 tokens : cela réduit la longueur de séquence utilisée par l'attention croisée, pour limiter le coût de calcul, en considérant que l'essentiel de l'information utile d'un compte rendu se trouve dans ses premiers tokens.

Cellule 13 - Reconstruction du jeu de données d'entraînement pour l'attention croisée

CE QUE FAIT CETTE CELLULE

Cette cellule relit toutes les représentations sauvegardées du jeu d'entraînement et les assemble en un jeu de données PyTorch prêt pour l'attention croisée.

LE CODE

```
cross_att_X_train_img = np.array(
    [load_saved_images(index, "train") for index in tqdm(range(len(X_train)))]])
cross_att_X_train_txt = np.array(
    [load_saved_texts(index, "train") for index in tqdm(range(len(X_train)))]])
cross_att_y_train = y_train.to_numpy()
cross_att_X_train_txt_tensor = torch.from_numpy(cross_att_X_train_txt).float()
cross_att_X_train_img_tensor = torch.from_numpy(cross_att_X_train_img).float()
cross_att_y_train_tensor = torch.from_numpy(cross_att_y_train).float()
cross_att_train_dataset = TensorDataset(
    cross_att_X_train_img_tensor, cross_att_X_train_txt_tensor, cross_att_y_train_tensor)
```

EXPLICATION DÉTAILLÉE

- Grâce aux fichiers sauvegardés à la cellule 7, cette étape se contente de relire des tenseurs déjà calculés, sans repasser les images et les textes dans les encodeurs - un gain de temps important par rapport à la partie B, où l'extraction des représentations pour l'attention croisée recalculait tout depuis les images brutes.
- Le reste de la cellule est identique dans sa logique à la partie B (cellule 16) : conversion en tenseurs PyTorch puis regroupement en TensorDataset.

Cellule 14 - Reconstruction du jeu de test et préparation des chargeurs

CE QUE FAIT CETTE CELLULE

Cette cellule reproduit la même reconstruction pour le jeu de test, puis prépare les chargeurs de lots pour l'entraînement et la validation.

LE CODE

```
cross_att_X_test_img = np.array(
    [load_saved_images(index, "test") for index in tqdm(range(len(X_test)))]])
cross_att_X_test_txt = np.array(
    [load_saved_texts(index, "test") for index in tqdm(range(len(X_test)))]])
cross_att_y_test = y_test.to_numpy()

cross_att_X_test_txt_tensor = torch.from_numpy(cross_att_X_test_txt).float()
cross_att_X_test_img_tensor = torch.from_numpy(cross_att_X_test_img).float()
cross_att_y_test_tensor = torch.from_numpy(cross_att_y_test).float()
cross_att_test_dataset = TensorDataset(
    cross_att_X_test_img_tensor, cross_att_X_test_txt_tensor, cross_att_y_test_tensor)

cross_att_train_loader = DataLoader(cross_att_train_dataset, batch_size=64, shuffle=True)
cross_att_val_loader = DataLoader(cross_att_test_dataset, batch_size=64, shuffle=False)
```

EXPLICATION DÉTAILLÉE

- Comme dans la partie B, le jeu d'entraînement est mélangé à chaque epoch (**shuffle=True**) tandis que le jeu de test garde un ordre fixe (**shuffle=False**), avec des lots de 64 exemples dans les deux cas.

- Cette fois, l'entraînement portera sur l'intégralité des 28 800 paires d'entraînement (et non sur un échantillon réduit comme pour l'attention croisée BiomedCLIP), grâce au gain d'efficacité permis par la sauvegarde et la relecture des représentations.

Cellule 15 - Couche de projection visuelle

CE QUE FAIT CETTE CELLULE

Cette cellule définit une couche supplémentaire, propre à BioViL-T : une projection qui aligne la dimension des patches d'image sur celle des tokens de texte avant l'attention croisée.

LE CODE

```
class VisualProjectionLayer(nn.Module):
    def __init__(self, img_dim=128, text_dim=768):
        super().__init__()
        # This layer bridges the dimensionality gap (128 -> 768)
        self.projector = nn.Linear(img_dim, text_dim)

    def forward(self, img_patches):
        # img_patches is [128, 14, 14]
        # Move channels to the back: [14, 14, 128]
        x = img_patches.permute(0, 2, 3, 1)

        # Flatten spatial dimensions: [196, 128]
        x = x.flatten(1, 2)

        # 3. Project to text dimension: [196, 768]
        projected_visual_tokens = self.projector(x)
        return projected_visual_tokens
```

EXPLICATION DÉTAILLÉE

- Les patches d'image de BioViL-T ont 128 dimensions, alors que les tokens de texte de CXR-BERT en ont 768 - contrairement à BiomedCLIP, où image et texte partageaient déjà la même dimension de 768. Cette couche est donc nécessaire ici pour permettre à l'attention croisée de comparer les deux.
- **permute(0, 2, 3, 1)** réorganise les dimensions du tenseur de patches, qui arrive au format [canaux, hauteur, largeur], vers un format [hauteur, largeur, canaux], plus adapté à la suite du traitement.
- **flatten(1, 2)** aplatit la grille spatiale 14x14 en une simple séquence de 196 patches, dans le même esprit que la séquence de patches utilisée pour BiomedCLIP.
- **self.projector**, une simple couche linéaire, transforme chaque patche de 128 dimensions en un vecteur de 768 dimensions, directement comparable aux tokens de texte.

À RETENIR - Pourquoi cette différence avec BiomedCLIP ?

BiomedCLIP a été conçu dès l'origine avec des dimensions internes identiques pour l'image et le texte (768), ce qui permettait à l'attention croisée de la partie B de comparer directement les deux. BioViL-T, optimisé différemment, produit des patches d'image à 128 dimensions : la couche de projection comble cet écart avant que l'attention croisée ne puisse opérer.

Cellule 16 - Architecture du modèle d'attention croisée BioViL-T

CE QUE FAIT CETTE CELLULE

Cette cellule définit l'architecture complète du modèle retenu pour le projet : la même logique d'attention croisée que pour BiomedCLIP, précédée de la couche de projection visuelle définie à la cellule précédente.

LE CODE

```
class CrossAttentionClassifierBiovil(nn.Module):
    def __init__(self, embed_dim=768, num_heads=8, dropout=0.3):
        super().__init__()

        self.projection_layer = VisualProjectionLayer()

        self.cross_attention = nn.MultiheadAttention(
            embed_dim=embed_dim,
            num_heads=num_heads,
            dropout=dropout,
            batch_first=True
        )

        self.layer_norm1 = nn.LayerNorm(embed_dim)
        self.layer_norm2 = nn.LayerNorm(embed_dim)

        self.ffn = nn.Sequential(
            nn.Linear(embed_dim, embed_dim * 2),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(embed_dim * 2, embed_dim)
        )

        self.classifier = nn.Sequential(
            nn.Linear(embed_dim, 256),
            nn.GELU(),
            nn.LayerNorm(256),
            nn.Dropout(dropout),
            nn.Linear(256, 1)
        )

    def forward(self, img_patches, text_tokens):
        img_patches_proj = self.projection_layer(img_patches)
        norm_text = self.layer_norm1(text_tokens)
        # Cross-Attention Core
        attn_output, _ = self.cross_attention(
            query=norm_text,
            key=img_patches_proj,
            value=img_patches_proj
        )
        x = (attn_output + text_tokens)

        # FFN stabilization step
        x = self.layer_norm2(self.ffn(x)) + x
```

```
# Max-pooling across sequence dimension
x_pooled, _ = torch.max(x, dim=1)
return self.classifier(x_pooled)
```

EXPLICATION DÉTAILLÉE

- La seule différence structurelle avec CrossAttentionClassifier (partie B, cellule 17) est l'ajout de **self.projection_layer**, appelée en tout début de forward pour ramener les patches d'image à 768 dimensions avant l'attention croisée elle-même.
- Le reste de l'architecture - attention multi-têtes, bloc feed-forward de stabilisation, max-pooling puis classifieur final - reproduit exactement la même logique que pour BiomedCLIP, ce qui permet d'attribuer les différences de performance entre les deux notebooks principalement à la qualité des représentations produites par chaque encodeur, plutôt qu'à la tête de classification elle-même.

À RETENIR - Le modèle retenu pour le projet

C'est cette architecture, combinant l'encodeur BioViL-T et cette tête d'attention croisée, qui constitue le modèle finalement retenu pour l'Auditeur de Cohérence Médicale, et qui est utilisé par l'application de démonstration.

Cellule 17 - Fonction d'entraînement avec arrêt anticipé

CE QUE FAIT CETTE CELLULE

Cette cellule définit la fonction d'entraînement du modèle retenu. Elle reprend la même structure générale que pour BiomedCLIP, avec un calendrier de taux d'apprentissage différent.

LE CODE

```
def cross_attention_train_biovil(model, train_loader, val_loader, criterion, optimizer,
                                scheduler, epochs=100, patience=3):
    history = {'loss': [], 'val_loss': [], 'accuracy': [], 'val_accuracy': []}
    best_val_loss = float('inf')
    patience_counter = 0
    best_model_weights = None
    best_epoch = 0

    for epoch in range(epochs):
        model.train()
        total_loss, correct = 0, 0

        for inputs_img, inputs_txt, labels in train_loader:
            inputs_img = inputs_img.to(device)
            inputs_txt = inputs_txt.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs_img, inputs_txt).squeeze(1)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
```

```
total_loss += loss.item()
correct += ((torch.sigmoid(outputs) >= 0.5).float() == labels).sum().item()

train_loss = total_loss / len(train_loader)
train_acc = correct / len(train_loader.dataset)

model.eval()
val_loss, val_correct = 0, 0
with torch.no_grad():
    for inputs_img, inputs_txt, labels in val_loader:
        inputs_img = inputs_img.to(device)
        inputs_txt = inputs_txt.to(device)
        labels = labels.to(device)
        outputs = model(inputs_img, inputs_txt).squeeze(1)
        loss = criterion(outputs, labels)
        val_loss += loss.item()
        val_correct += ((torch.sigmoid(outputs) >= 0.5).float() == labels).sum().item()

val_loss /= len(val_loader)
val_acc = val_correct / len(val_loader.dataset)

# Track metrics so they show up in Hugging Face / MLflow
#mlflow.log_metric("train_loss", train_loss, step=epoch)
#mlflow.log_metric("val_loss", val_loss, step=epoch)

#scheduler.step()
scheduler.step(val_loss)
current_lr = optimizer.param_groups[0]['lr']

history['loss'].append(train_loss)
history['val_loss'].append(val_loss)
history['accuracy'].append(train_acc)
history['val_accuracy'].append(val_acc)

print(f"Epoch [{epoch+1}/{epochs}], Loss: {train_loss:.4f}, Acc: {train_acc:.4f}, "
      f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}, LR: {current_lr:.6f}")

if val_loss < best_val_loss:
    best_val_loss = val_loss
    patience_counter = 0
    best_epoch = epoch + 1
    best_model_weights = copy.deepcopy(model.state_dict())
else:
    patience_counter += 1
    print(f"-> Validation loss did not improve. Patience: {patience_counter}/{patience}")

if patience_counter >= patience:
    print(f"Early stopping triggered! Stopping training at epoch {epoch+1}.")
    break

if best_model_weights is not None:
    print(f"Restoring best model weights found at Epoch {best_epoch} "
```

```
f"(Val Loss: {best_val_loss:.4f})")
model.load_state_dict(best_model_weights)

return history
```

EXPLICATION DÉTAILLÉE

- La structure générale (boucle d'entraînement, boucle de validation, arrêt anticipé après 3 epochs sans amélioration, restauration des meilleurs poids) est la même que pour l'attention croisée BiomedCLIP (partie B, cellule 18).
- La différence porte sur le calendrier de taux d'apprentissage : ici, **scheduler.step(val_loss)** transmet la perte de validation au planificateur, qui décide de réduire le taux d'apprentissage seulement si cette perte stagne (logique détaillée à la cellule suivante), alors que la partie B utilisait une décroissance programmée à l'avance, indépendante des résultats observés.
- Les lignes commentées **#mlflow.log_metric(...)** montrent que cette fonction a initialement servi, en conditions réelles d'entraînement, à journaliser chaque epoch dans un serveur de suivi d'expériences MLflow ; ces lignes sont conservées en commentaire mais ne s'exécutent pas dans cette version du notebook.

Cellule 18 - Instanciation du modèle, de la perte et de l'optimiseur

CE QUE FAIT CETTE CELLULE

Cette cellule crée une instance du modèle retenu et configure ses outils d'entraînement, avec un calendrier de taux d'apprentissage adaptatif plutôt que programmé à l'avance.

LE CODE

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

cross_att_model = CrossAttentionClassifierBiovil().to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.AdamW(cross_att_model.parameters(), lr=1e-5, weight_decay=1e-2)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=2)

# Clear cache just to be safe
if torch.cuda.is_available():
    torch.cuda.empty_cache()
```

EXPLICATION DÉTAILLÉE

- La fonction de perte (BCEWithLogitsLoss) et l'optimiseur (AdamW, même taux d'apprentissage de 1e-5) sont identiques à ceux utilisés pour BiomedCLIP.
- **ReduceLROnPlateau(mode='min', factor=0.5, patience=2)** est un calendrier adaptatif : il surveille la perte de validation, et si elle ne s'améliore pas pendant 2 epochs consécutives, il divise le taux d'apprentissage par deux (factor=0.5). Contrairement à la décroissance en cosinus utilisée pour BiomedCLIP, qui suit une courbe fixée à l'avance, celui-ci réagit aux résultats réellement observés pendant l'entraînement.

À RETENIR - Calendrier programmé ou calendrier adaptatif ?

CosineAnnealingLR (partie B) réduit le taux d'apprentissage selon une courbe connue à l'avance, quel que soit le déroulement de l'entraînement. ReduceLROnPlateau, utilisé ici, attend au contraire que la progression stagne réellement avant d'agir. Les deux approches sont courantes ; le choix dépend souvent de l'expérience acquise sur un type de modèle donné.

Cellule 19 - Résumé de l'architecture du modèle retenu

CE QUE FAIT CETTE CELLULE

Cette cellule affiche un résumé de l'architecture du modèle d'attention croisée BioViL-T et de son nombre de paramètres.

LE CODE

```
summary(cross_att_model)
```

EXPLICATION DÉTAILLÉE

- Comme dans la partie B, ce résumé permet de vérifier la cohérence de l'architecture (notamment la présence de la couche de projection supplémentaire) avant de lancer l'entraînement.

Cellule 20 - Lancement de l'entraînement du modèle retenu

CE QUE FAIT CETTE CELLULE

Cette cellule lance l'entraînement du modèle d'attention croisée BioViL-T, sur l'intégralité des 28 800 paires d'entraînement.

LE CODE

```
# Launch the training block
history = cross_attention_train_biovil(
    model=cross_att_model,
    train_loader=cross_att_train_loader,
    val_loader=cross_att_val_loader,
    criterion=criterion,
    optimizer=optimizer,
    scheduler = scheduler,
    epochs=50
)
```

EXPLICATION DÉTAILLÉE

- Cet appel suit le même schéma que pour BiomedCLIP, avec la fonction et les outils spécifiques à BioViL-T préparés aux cellules précédentes.

À RETENIR - Résultat obtenu

L'entraînement s'arrête par arrêt anticipé à l'epoch 23 (sur 50 prévues), les meilleurs poids étant ceux de l'epoch 20 (perte de validation environ 0,28). À cette epoch, l'exactitude d'entraînement atteint environ 0,91 et l'exactitude de validation environ 0,88 - une nette progression par rapport à la similarité seule (0,78), et par rapport aux meilleures approches BiomedCLIP (0,82 à 0,84).

Cellule 21 - Visualisation de la courbe d'apprentissage

CE QUE FAIT CETTE CELLULE

Cette cellule trace l'évolution de la perte d'entraînement et de validation au fil des epochs.

LE CODE

```
# Plot the training and validation loss
plt.plot(history["loss"], label="Training Loss")
plt.plot(history["val_loss"], label="Validation Loss")
plt.title("Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

EXPLICATION DÉTAILLÉE

- Cette cellule est identique dans sa logique à celles des parties précédentes, appliquée ici à l'historique d'entraînement du modèle retenu.

Cellule 22 - Fonction d'évaluation finale

CE QUE FAIT CETTE CELLULE

Cette cellule définit une nouvelle fonction d'évaluation, qui calcule les métriques directement à partir des prédictions du modèle entraîné, plutôt qu'à partir d'un score de similarité comparé à un seuil.

LE CODE

```
def bin_classif_metrics_cross_att(predictions, targets, probabilities, type='Test'):

    accuracy = accuracy_score(targets, predictions)
    precision = precision_score(targets, predictions)
    recall = recall_score(targets, predictions)
    f1 = f1_score(targets, predictions)

    tn, fp, fn, tp = confusion_matrix(targets, predictions).ravel()
    specificity = tn / (tn + fp)

    print(f"{type} Accuracy: {accuracy:.4f}")
    print(f"{type} Precision: {precision:.4f}")
    print(f"{type} Recall: {recall:.4f}")
```

```
print(f"{type} F1-Score: {f1:.4f}")
print(f"{type} Specificity: {specificity:.4f} (FPR is {1-specificity:.4f})")
print(f"{type} ROC-AUC Score: {roc_auc_score(targets, probabilities):.4f}")
```

EXPLICATION DÉTAILLÉE

- Cette fonction ressemble à `bin_classif_metrics` (cellule 9), mais prend directement en entrée des **prédictions** déjà calculées par le réseau de neurones (0 ou 1), plutôt qu'un score de similarité à comparer à un seuil choisi séparément - une différence logique, puisque le modèle d'attention croisée produit lui-même sa décision finale, via sa couche de classification.
- Le **ROC-AUC** est calculé à partir des probabilités brutes (probabilities), pas des décisions finales, pour rester une mesure indépendante de tout seuil de décision.

Cellule 23 - Calcul des prédictions finales sur le jeu de test

CE QUE FAIT CETTE CELLULE

Cette cellule fait passer l'ensemble du jeu de test dans le modèle entraîné (restauré à ses meilleurs poids), et collecte les prédictions, les étiquettes réelles et les probabilités associées.

LE CODE

```
# Validation's Metrics
all_predictions = []
all_targets = []
all_probabilities = []
cross_att_model.to(device)
cross_att_model.eval()
with torch.no_grad():
    for inputs_img, inputs_txt, labels in cross_att_val_loader:
        inputs_img = inputs_img.to(device)
        inputs_txt = inputs_txt.to(device)

        outputs = cross_att_model(inputs_img, inputs_txt).squeeze(1)
        predictions = (torch.sigmoid(outputs) >= 0.5)
        all_predictions.extend(predictions.cpu().numpy())
        all_targets.extend(labels.cpu().numpy())
        all_probabilities.extend(outputs.cpu().numpy())

all_predictions = np.array(all_predictions, dtype=int)
all_targets = np.array(all_targets, dtype=int)
all_probabilities = np.array(all_probabilities)
```

EXPLICATION DÉTAILLÉE

- **`cross_att_model.eval()`** et **`torch.no_grad()`** assurent que cette évaluation se fait sans aucune mise à jour des poids du modèle, exactement comme une utilisation réelle du modèle en production.
- La boucle parcourt tous les lots du jeu de test, calcule la sortie du modèle pour chacun, applique un seuil de décision fixe de 0,5 sur la probabilité (après passage dans une fonction sigmoïde), et accumule prédictions, étiquettes réelles et probabilités brutes dans trois listes.
- Ces listes sont enfin converties en tableaux numpy, prêts pour le calcul final des métriques à la cellule suivante.

À RETENIR - Pourquoi un seuil fixe de 0,5 ici, et pas un seuil optimisé comme pour la similarité ?

Le modèle d'attention croisée produit une sortie déjà entraînée pour se rapprocher de 0 ou de 1 ; un seuil de décision à 0,5, la valeur par défaut d'une classification binaire équilibrée, est donc un choix naturel, contrairement à la similarité cosinus brute de la première approche, dont l'échelle n'était pas calibrée pour une décision binaire directe.

Cellule 24 - Affichage des métriques finales du modèle retenu

CE QUE FAIT CETTE CELLULE

Cette dernière cellule de calcul affiche les métriques finales du modèle retenu pour le projet, sur le jeu de test.

LE CODE

```
# Validation set Metrics
bin_classif_metrics_cross_att(all_predictions, all_targets, all_probabilities)
```

EXPLICATION DÉTAILLÉE

- Cet appel applique la fonction définie à la cellule 22 aux résultats collectés à la cellule 23, et affiche le bilan complet de performance du modèle retenu.

À RETENIR - Résultat final du projet

Sur le jeu de test, le modèle retenu (BioViL-T avec attention croisée) atteint une exactitude de 0,8842, une précision de 0,8760, un rappel de 0,8950, un F1 de 0,8854, une spécificité de 0,8733 et un ROC-AUC de 0,9505. C'est la meilleure performance obtenue parmi toutes les approches explorées dans le projet, ce qui justifie son choix comme modèle de production pour l'application de démonstration.

Cellule 25 - Code de référence pour le suivi MLflow (non exécuté)

CE QUE FAIT CETTE CELLULE

Cette dernière cellule du notebook est un bloc de code mis en commentaire (entre triples guillemets), conservé à titre de référence : il montre comment l'entraînement du modèle retenu a été suivi et journalisé via MLflow lors des entraînements réels.

LE CODE

```
"""
EXPERIMENT_NAME="model_biovil_cross_attention_mlp"

# Set tracking URI to Hugging Face
mlflow.set_tracking_uri(os.environ["APP_URI"])

# Look for the experiment, create it if missing, and activate it
mlflow.set_experiment(EXPERIMENT_NAME)
```

```

with mlflow.start_run():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    cross_att_model = CrossAttentionClassifierBiovil().to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.AdamW(cross_att_model.parameters(), lr=1e-5, weight_decay=1e-2)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5,
    patience=2)

    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    # Log hyperparams explicitly
    mlflow.log_params({
        "lr": 1e-5,
        "weight_decay": 1e-2,
        "epochs": 50,
        "scheduler": "ReduceLROnPlateau"
    })

    # Launch the training block
    history = cross_attention_train_biovil(
        model=cross_att_model,
        train_loader=cross_att_train_loader,
        val_loader=cross_att_val_loader,
        criterion=criterion,
        optimizer=optimizer,
        scheduler = scheduler
    )

    # Right before exiting the context manager
    mlflow.pytorch.log_model(cross_att_model, name="biovil_cross_attention")
"""

```

EXPLICATION DÉTAILLÉE

- Les triples guillemets (""") au début et à la fin transforment tout ce bloc en une simple chaîne de caractères pour Python, qui ne produit aucun effet à l'exécution : c'est une façon courante de conserver du code à titre de documentation, sans qu'il s'exécute.
- **mlflow.set_tracking_uri(...)** et **mlflow.set_experiment(...)** connectent le suivi d'expérience à un serveur MLflow distant, et y créent ou activent une expérience nommée.
- **with mlflow.start_run():** ouvre une session de suivi : tout ce qui est journalisé à l'intérieur de ce bloc (hyperparamètres, métriques au fil de l'entraînement, modèle final) est associé à cette session, consultable ensuite dans l'interface MLflow.
- **mlflow.log_params({...})** enregistre les hyperparamètres utilisés (taux d'apprentissage, régularisation, nombre d'epochs, type de calendrier), pour pouvoir comparer plusieurs entraînements entre eux.
- **mlflow.pytorch.log_model(...)** sauvegarde le modèle entraîné lui-même dans MLflow, ce qui permet de le retrouver et de le recharger plus tard, par exemple pour le déployer.

À RETENIR - Pourquoi conserver du code qui ne s'exécute pas ?

Documenter la manière dont l'entraînement réel a été suivi et tracé, même si ce suivi n'est pas reproduit dans cette version du notebook, est une bonne pratique de transparence : elle montre que le choix du modèle final repose sur un suivi rigoureux des expériences, et pas seulement sur l'exécution ponctuelle visible dans ce notebook.

Synthèse - le modèle retenu

Le tableau ci-dessous compare les deux approches BioViL-T, et situe le modèle finalement retenu par rapport à la meilleure approche BiomedCLIP de la partie B.

Approche	ROC-AUC (test)	Exactitude (test)	F1 (test)
Régression logistique BiomedCLIP	0,83	0,75	0,75
Similarité BioViL-T	0,86	0,78	0,78
Attention croisée BioViL-T (retenu)	0,95	0,88	0,89

À RETENIR - Ce qu'il faut retenir de la comparaison

Deux facteurs expliquent la performance du modèle retenu : un encodeur spécialisé pour la radiographie thoracique plutôt qu'un encodeur biomédical généraliste, et une fusion fine par attention croisée plutôt qu'une simple comparaison de vecteurs globaux. Ces deux choix se renforcent l'un l'autre, et c'est leur combinaison qui constitue le modèle de production de l'Auditeur de Cohérence Médicale.